Al-Balqa Applied University
Faculty of Engineering Technology
Communication Technology Department



Communications and data transmission
Chapter 2 Network Models

Prof. Majed Dwairi
majeddw@bau.edu.jo
majeddw@gmail.com

Network Models

The second chapter is a preparation for the rest of the book. The next five parts of the book is devoted to one of the layers in the TCP/IP protocol suite. In this chapter, we first discuss the idea of network models in general and the TCP/IP protocol suite in particular.

Two models have been devised to define computer network operations: the TCP/IP protocol suite and the OSI model. In this chapter, we first discuss a general subject, protocol layering, which is used in both models. We then concentrate on the TCP/IP protocol suite, on which the book is based. The OSI model is briefly discuss for comparison with the TCP/IP protocol suite.

2.1 PROTOCOL LAYERING

We defined the term *protocol* in Chapter 1. In data communication and networking, a protocol defines the rules that both the sender and receiver and all intermediate devices need to follow to be able to communicate effectively. When communication is simple, we may need only one simple protocol; when the communication is complex, we may need to divide the task between different layers, in which case we need a protocol at each layer, or **protocol layering.**

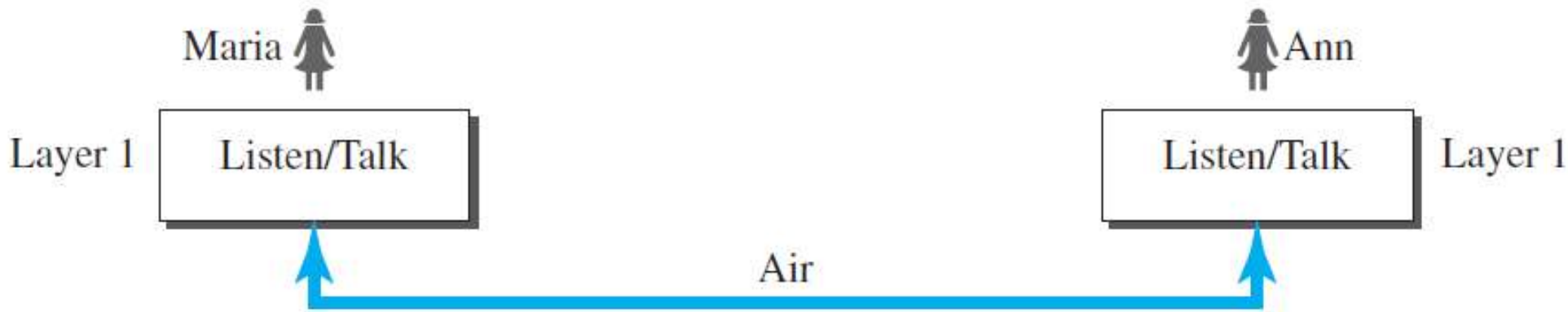
2.1.1 Scenarios

Let us develop two simple scenarios to better understand the need for protocol layering.

First Scenario

In the first scenario, communication is so simple that it can occur in only one layer. Assume Maria and Ann are neighbors with a lot of common ideas. Communication between Maria and Ann takes place in one layer, face to face, in the same language, as shown in Figure 2.1.

Figure 2.1 *A single-layer protocol*



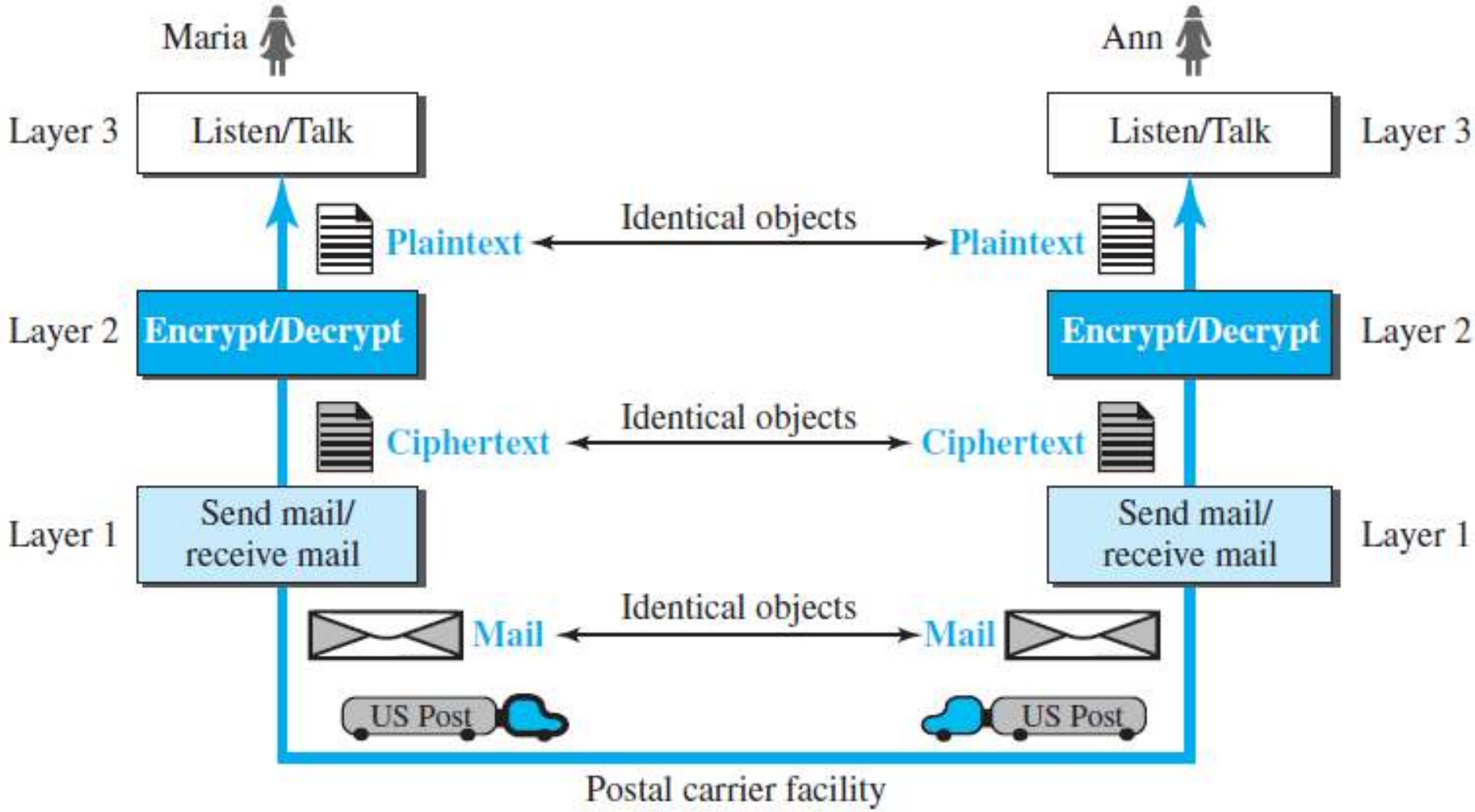
Even in this simple scenario, we can see that a set of rules needs to be followed. First, Maria and Ann know that they should greet each other when they meet. Second, they know that they should confine their vocabulary to the level of their friendship. Third, each party knows that she should refrain from speaking when the other party is speaking. Fourth, each party knows that the conversation should be a dialog, not a monolog: both should have the opportunity to talk about the issue. Fifth, they should exchange some nice words when they leave.

We can see that the protocol used by Maria and Ann is different from the communication between a professor and the students in a lecture hall. The communication in the second case is mostly monolog; the professor talks most of the time unless a student has a question, a situation in which the protocol dictates that she should raise her hand and wait for permission to speak. In this case, the communication is normally very formal and limited to the subject being taught.

Second Scenario

In the second scenario, we assume that Ann is offered a higher-level position in her company, but needs to move to another branch located in a city very far from Maria. The two friends still want to continue their communication and exchange ideas because they have come up with an innovative project to start a new business when they both retire. They decide to continue their conversation using regular mail through the post office. However, they do not want their ideas to be revealed by other people if the letters are intercepted. They agree on an encryption/decryption technique. The sender of the letter encrypts it to make it unreadable by an intruder; the receiver of the letter decrypts it to get the original letter. We discuss the encryption/decryption methods in Chapter 31, but for the moment we assume that Maria and Ann use one technique that makes it hard to decrypt the letter if one does not have the key for doing so. Now we can say that the communication between Maria and Ann takes place in three layers, as shown in Figure 2.2. We assume that Ann and Maria each have three machines (or robots) that can perform the task at each layer.

Figure 2.2 *A three-layer protocol*



Let us assume that Maria sends the first letter to Ann. Maria talks to the machine at the third layer as though the machine is Ann and is listening to her. The third layer machine listens to what Maria says and creates the plaintext (a letter in English), which is passed to the second layer machine. The second layer machine takes the plaintext, encrypts it, and creates the ciphertext, which is passed to the first layer machine. The first layer machine, presumably a robot, takes the ciphertext, puts it in an envelope, adds the sender and receiver addresses, and mails it.

At Ann's side, the first layer machine picks up the letter from Ann's mail box, recognizing the letter from Maria by the sender address. The machine takes out the ciphertext from the envelope and delivers it to the second layer machine. The second layer machine decrypts the message, creates the plaintext, and passes the plaintext to the third-layer machine. The third layer machine takes the plaintext and reads it as though Maria is speaking.

Protocol layering enables us to divide a complex task into several smaller and simpler tasks. For example, in Figure 2.2, we could have used only one machine to do the job of all three machines. However, if Maria and Ann decide that the encryption/decryption done by the machine is not enough to protect their secrecy, they would have to change the whole machine. In the present situation, they need to change only the second layer machine; the other two can remain the same. This is referred to as *modularity*. Modularity in this case means independent layers. A layer (module) can be defined as a black box with inputs and outputs, without concern about how inputs are changed to outputs. If two machines provide the same outputs when given the same inputs, they can replace each other. For example, Ann and Maria can buy the second layer machine from two different manufacturers. As long as the two machines create the same ciphertext from the same plaintext and vice versa, they do the job.

One of the advantages of protocol layering is that it allows us to separate the services from the implementation. A layer needs to be able to receive a set of services from the lower layer and to give the services to the upper layer; we don't care about how the layer is implemented. For example, Maria may decide not to buy the machine (robot) for the first layer; she can do the job herself. As long as Maria can do the tasks provided by the first layer, in both directions, the communication system works.

Another advantage of protocol layering, which cannot be seen in our simple examples but reveals itself when we discuss protocol layering in the Internet, is that communication does not always use only two end systems; there are intermediate systems that need only some layers, but not all layers. If we did not use protocol layering, we would have to make each intermediate system as complex as the end systems, which makes the whole system more expensive.

advantages of protocol layering

- is that it allows us to separate the services from the implementation. A layer needs to be able to receive a set of services from the lower layer and to give the services to the upper layer
- which cannot be seen in our simple examples but reveals itself when we discuss protocol layering in the Internet, is that communication does not always use only two end systems; there are intermediate systems that need only some layers, but not all layers

Is there any disadvantage to protocol layering? One can argue that having a single layer makes the job easier. There is no need for each layer to provide a service to the upper layer and give service to the lower layer. For example, Ann and Maria could find or build one machine that could do all three tasks. However, as mentioned above, if one day they found that their code was broken, each would have to replace the whole machine with a new one instead of just changing the machine in the second layer.

2.1.2 Principles of Protocol Layering

Let us discuss two principles of protocol layering.

First Principle

The first principle dictates that if we want bidirectional communication, we need to make each layer so that it is able to perform two opposite tasks, one in each direction. For example, the third layer task is to listen (in one direction) and *talk* (in the other direction). The second layer needs to be able to encrypt and decrypt. The first layer needs to send and receive mail.

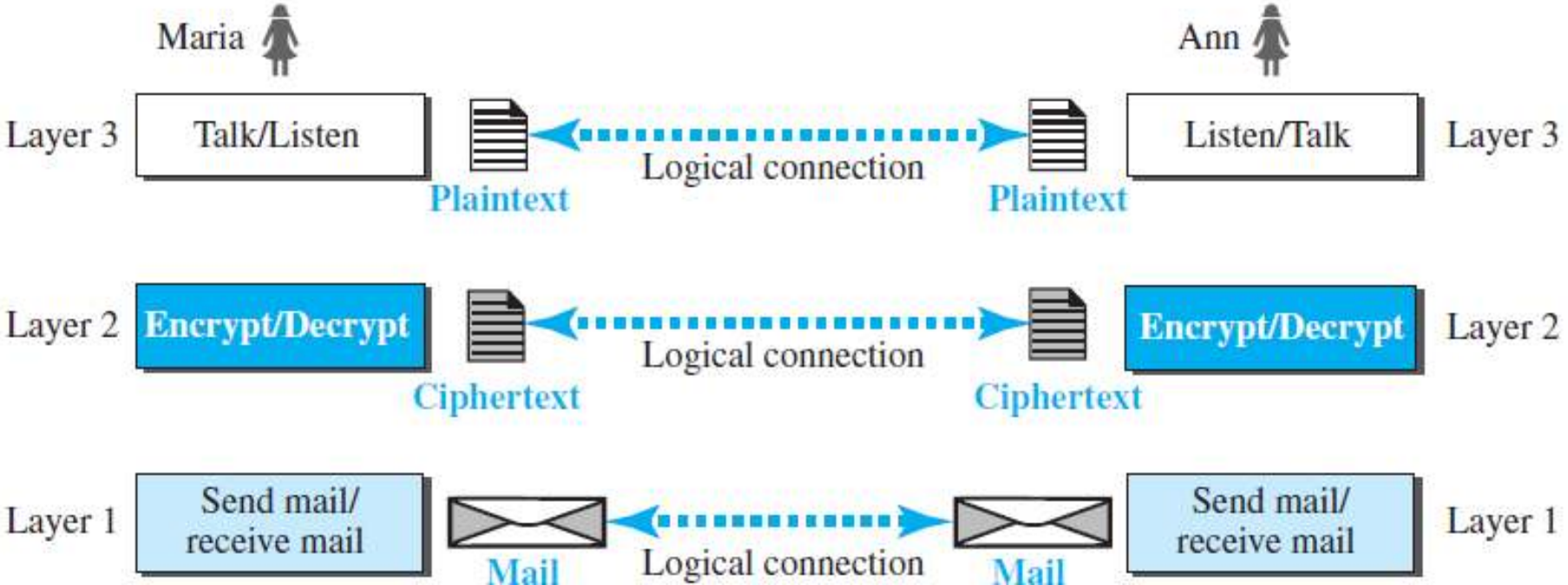
Second Principle

The second principle that we need to follow in protocol layering is that the two objects under each layer at both sites should be identical. For example, the object under layer 3 at both sites should be a plaintext letter. The object under layer 2 at both sites should be a ciphertext letter. The object under layer 1 at both sites should be a piece of mail.

2.1.3 Logical Connections

After following the above two principles, we can think about logical connection between each layer as shown in Figure 2.3. This means that we have layer-to-layer communication. Maria and Ann can think that there is a logical (imaginary) connection at each layer through which they can send the object created from that layer. We will see that the concept of logical connection will help us better understand the task of layering we encounter in data communication and networking.

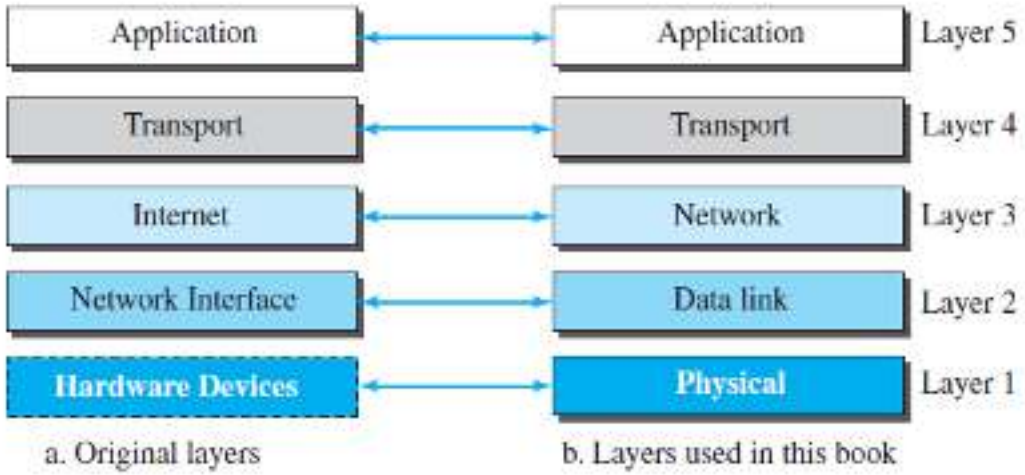
Figure 2.3 *Logical connection between peer layers*



2.2 TCP/IP PROTOCOL SUITE

Now that we know about the concept of protocol layering and the logical communication between layers in our second scenario, we can introduce the TCP/IP (Transmission Control Protocol/Internet Protocol). TCP/IP is a protocol suite (a set of protocols organized in different layers) used in the Internet today. It is a hierarchical protocol made up of interactive modules, each of which provides a specific functionality. The term *hierarchical* means that each upper level protocol is supported by the services provided by one or more lower level protocols. The original TCP/IP protocol suite was defined as four software layers built upon the hardware. Today, however, TCP/IP is thought of as a five-layer model. Figure 2.4 shows both configurations.

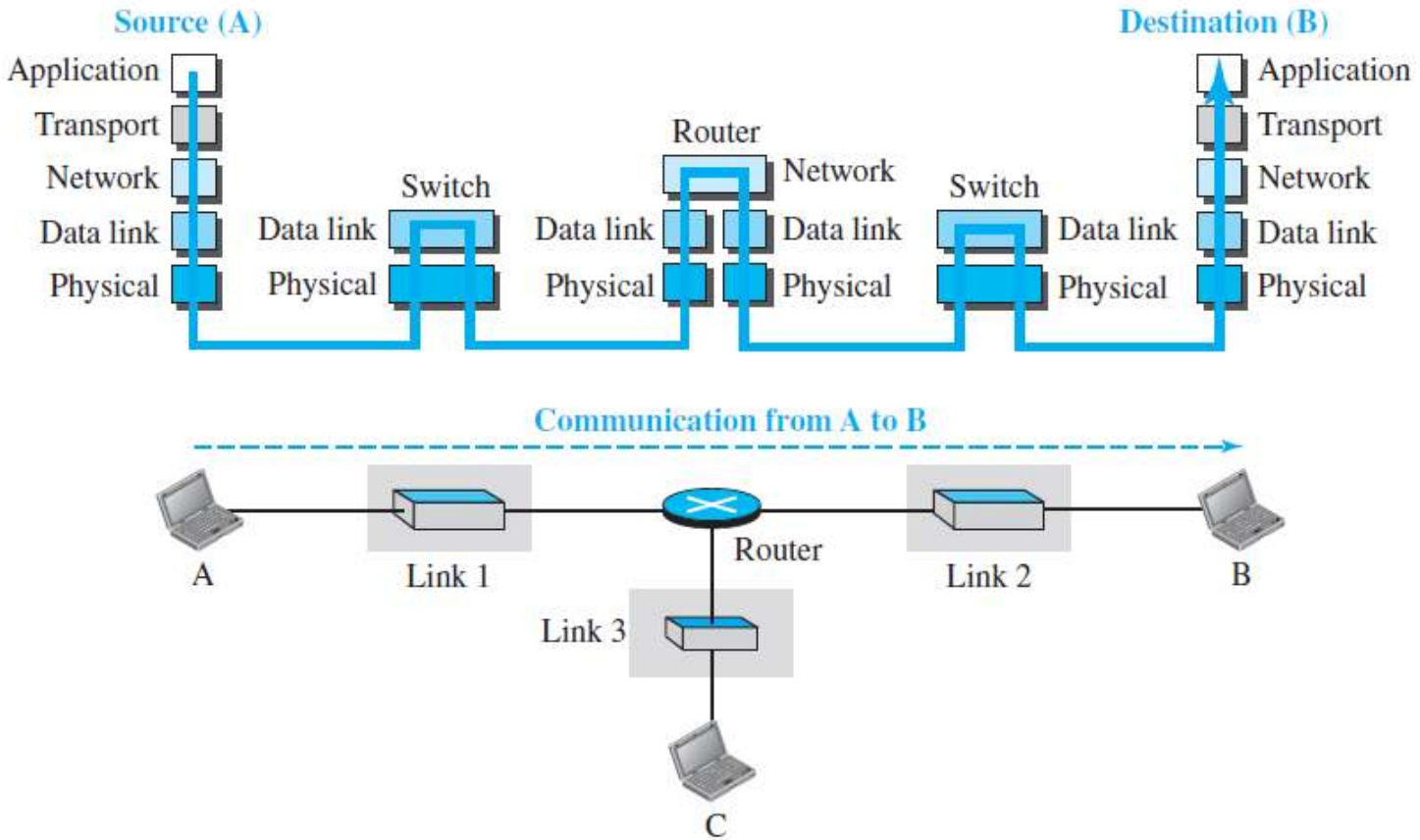
Figure 2.4 *Layers in the TCP/IP protocol suite*



2.2.1 Layered Architecture

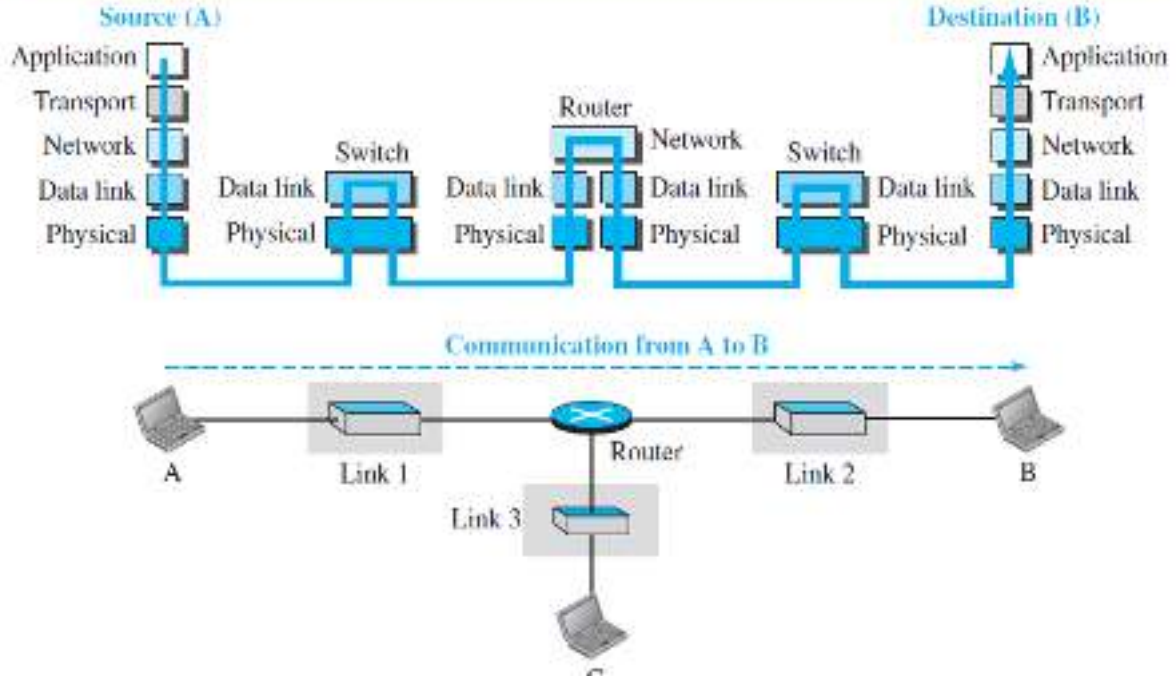
To show how the layers in the TCP/IP protocol suite are involved in communication between two hosts, we assume that we want to use the suite in a small internet made up of three LANs (links), each with a link-layer switch. We also assume that the links are connected by one router, as shown in Figure 2.5.

Figure 2.5 *Communication through an internet*



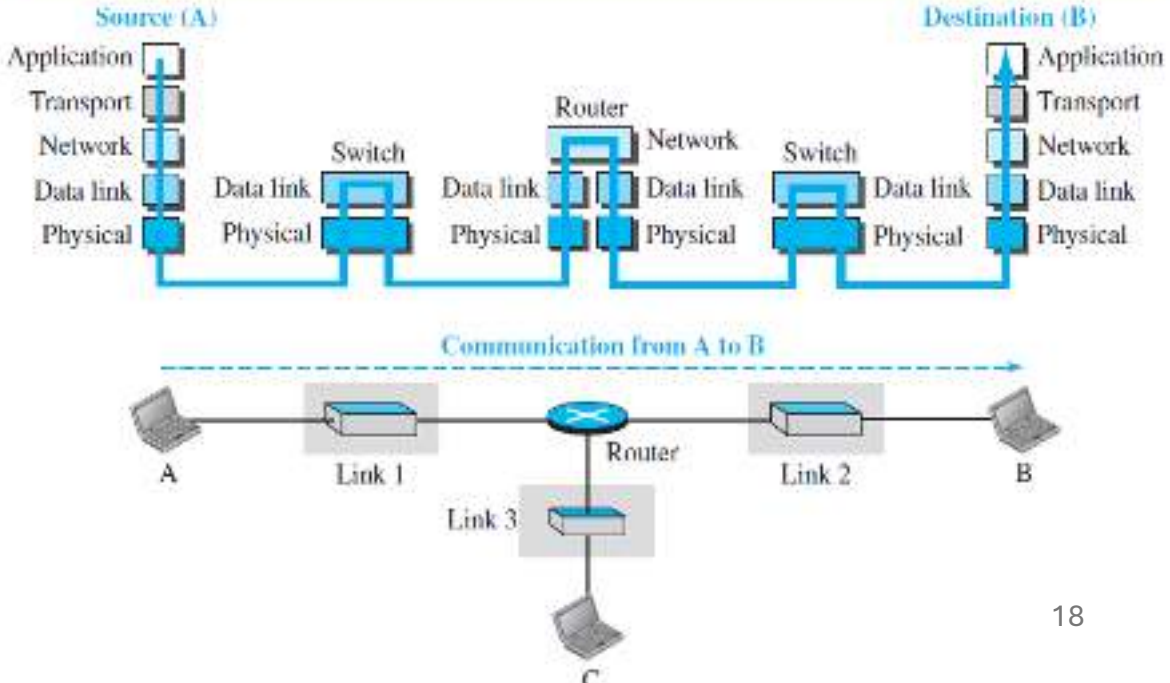
Let us assume that computer A communicates with computer B. As the figure shows, we have five communicating devices in this communication: source host (computer A), the link-layer switch in link 1, the router, the link-layer switch in link 2, and the destination host (computer B). Each device is involved with a set of layers depending on the role of the device in the internet. The two hosts are involved in all five layers; the source host needs to create a message in the application layer and send it down the layers so that it is physically sent to the destination host. The destination host needs to receive the communication at the physical layer and then deliver it through the other layers to the application layer.

Figure 2.5 *Communication through an internet*



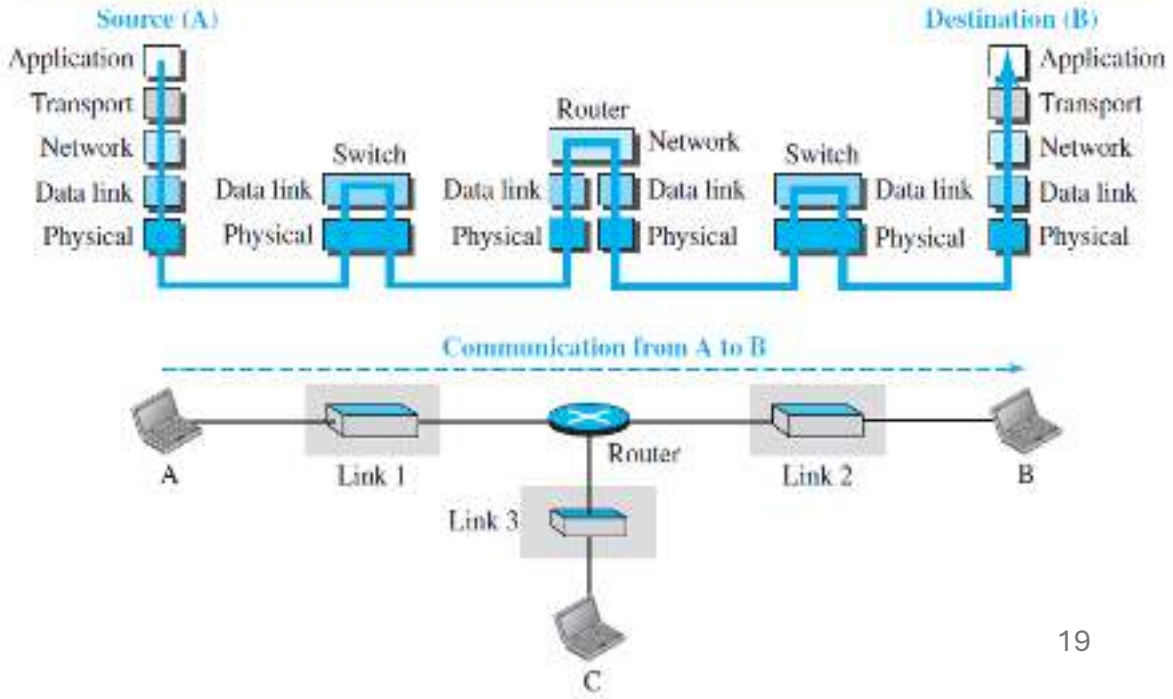
The router is involved in only three layers; there is no transport or application layer in a router as long as the router is used only for routing. Although a router is always involved in one network layer, it is involved in *n* combinations of link and physical layers in which *n* is the number of links the router is connected to. The reason is that each link may use its own data-link or physical protocol. For example, in the above figure, the router is involved in three links, but the message sent from source A to destination B is involved in two links. Each link may be using different link-layer and physical-layer protocols; the router needs to receive a packet from link 1 based on one pair of protocols and deliver it to link 2 based on another pair of protocols.

Figure 2.5 *Communication through an internet*



A link-layer switch in a link, however, is involved only in two layers, data-link and physical. Although each switch in the above figure has two different connections, the connections are in the same link, which uses only one set of protocols. This means that, unlike a router, a link-layer switch is involved only in one data-link and one physical

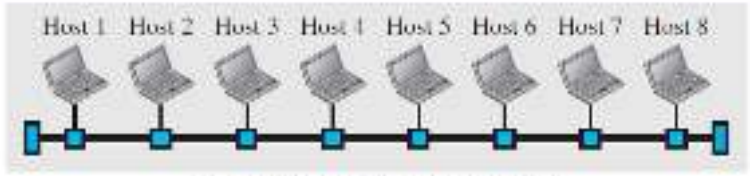
Figure 2.5 *Communication through an internet*



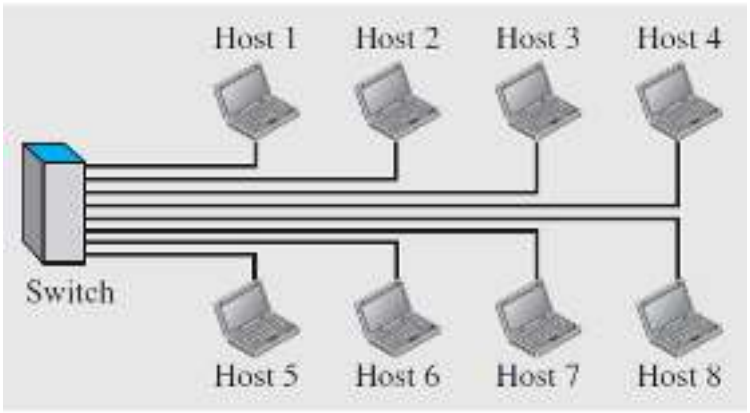
In the previous lecture

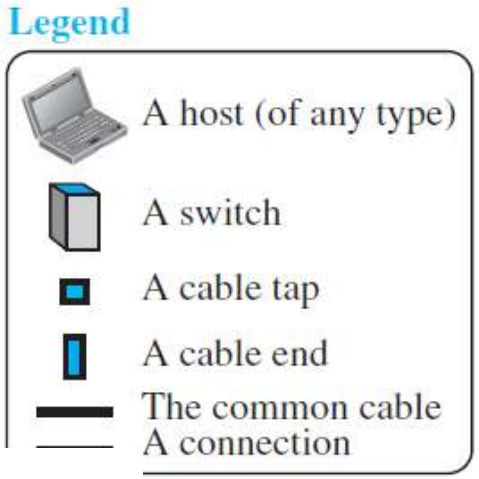
NETWORK TYPES
Local Area Network

Figure 1.8 *An isolated LAN in the past and today*



a. LAN with a common cable (past)





1.3.2 Wide Area Network

A **wide area network (WAN)** is also an interconnection of devices capable of communication. However, there are some differences between a LAN and a WAN. A LAN is normally

LAN

- is normally limited in size, spanning an office, a building, or a campus;
- interconnects hosts
- Is normally privately owned by the organization that uses it

WAN

- has a wider geographical span, spanning a town, a state, a country, or even the world.
- interconnects connecting devices such as switches, routers, or modems
- is normally created and run by communication companies and leased by an organization that uses it

Point-to-Point WAN



Figure 1.10 *A switched WAN*

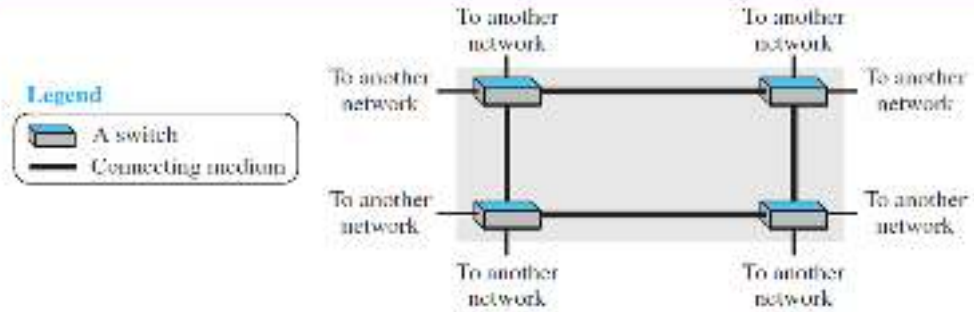
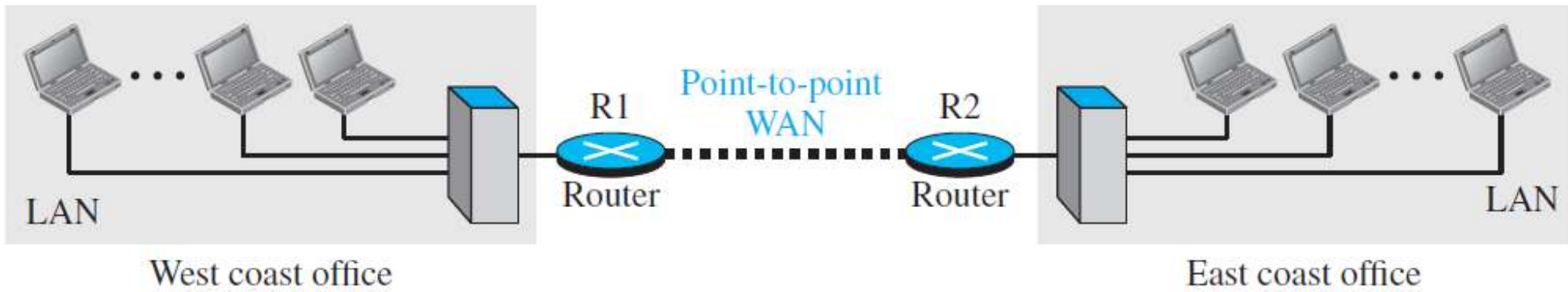


Figure 1.11 *An internetwork made of two LANs and one point-to-point WAN*



Switching

An internet is a **switched network** in which a switch connects at least two links together.

Figure 1.13 *A circuit-switched network*

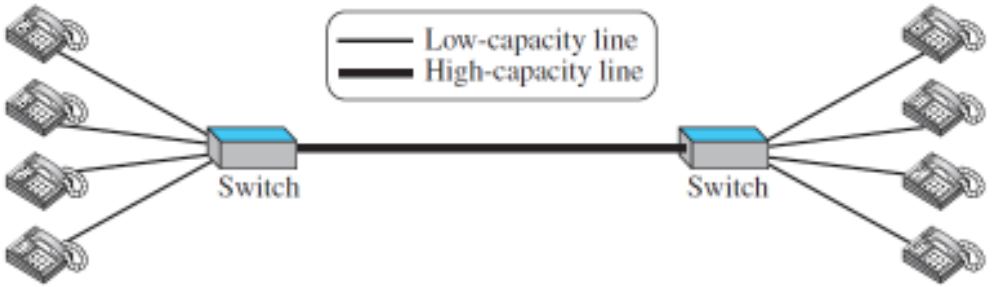
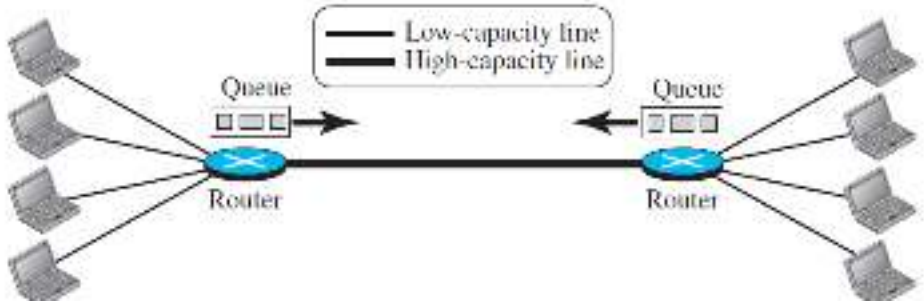
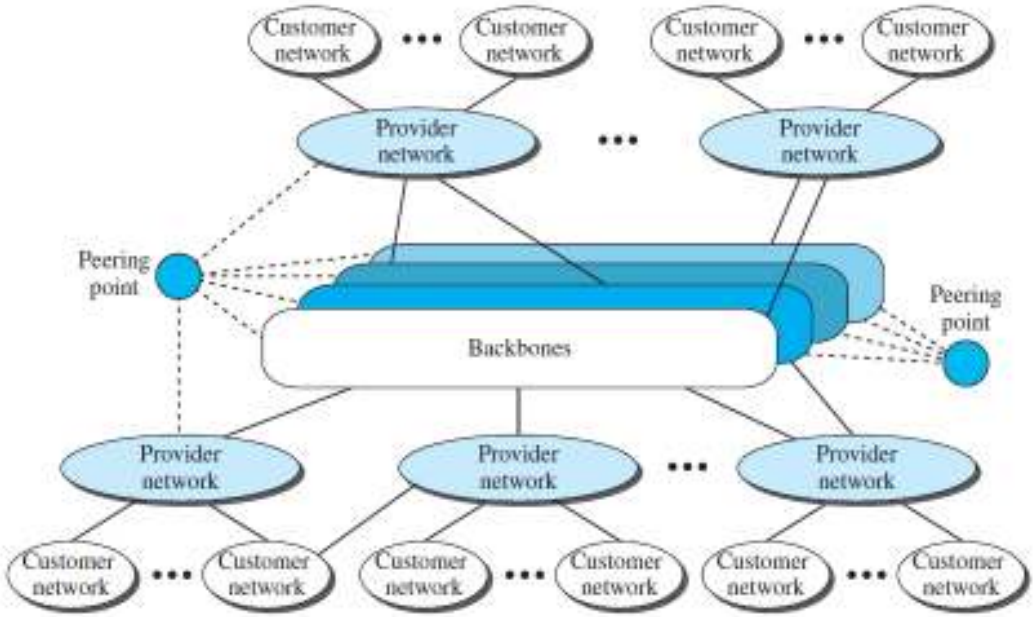


Figure 1.14 *A packet-switched network*



The Internet

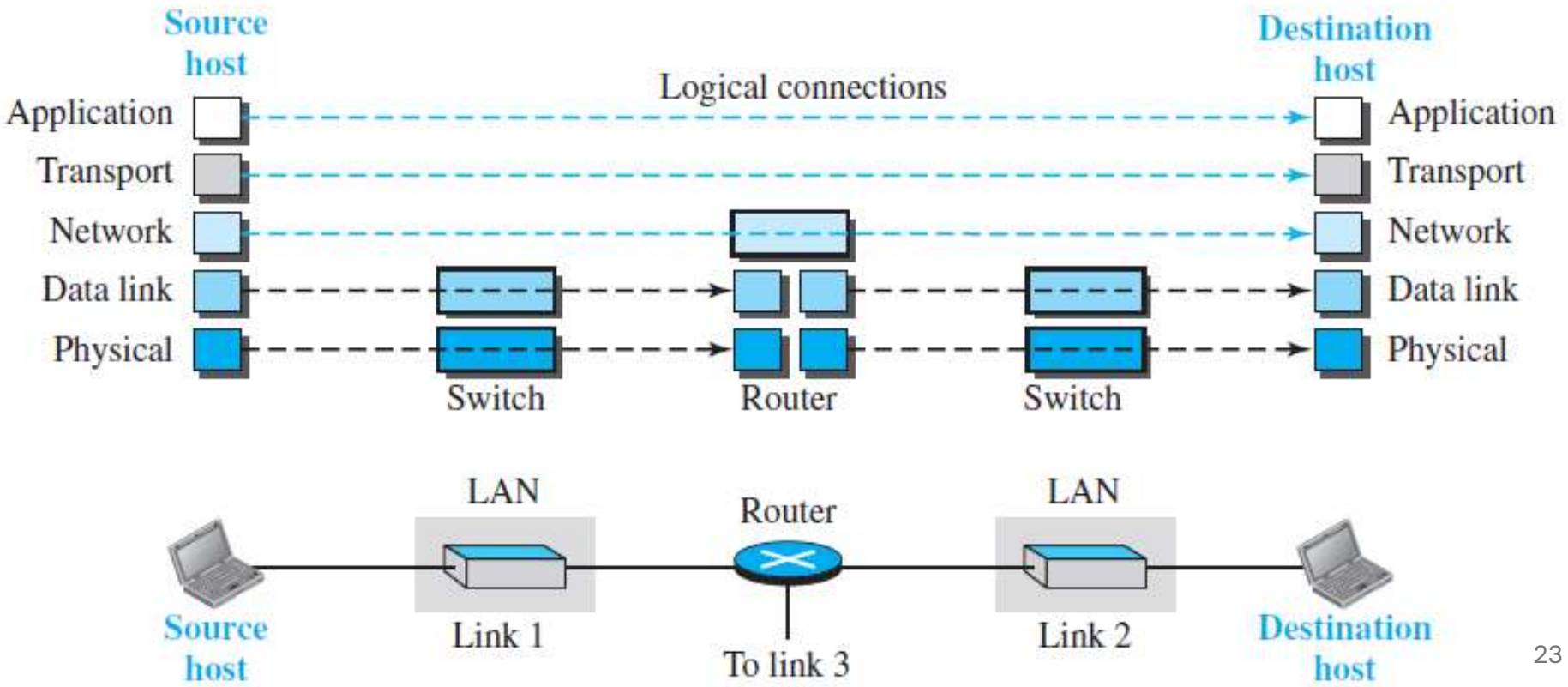
Figure 1.15 *The Internet today*



2.2.2 Layers in the TCP/IP Protocol Suite

After the above introduction, we briefly discuss the functions and duties of layers in the TCP/IP protocol suite. Each layer is discussed in detail in the next five parts of the book. To better understand the duties of each layer, we need to think about the logical connections between layers. Figure 2.6 shows logical connections in our simple internet.

Figure 2.6 *Logical connections between layers of the TCP/IP protocol suite*



Using logical connections makes it easier for us to think about the duty of each layer. As the figure shows, the duty of the application, transport, and network layers is end-to-end. However, the duty of the data-link and physical layers is hop-to-hop, in which a hop is a host or router. In other words, the domain of duty of the top three layers is the internet, and the domain of duty of the two lower layers is the link.

Another way of thinking of the logical connections is to think about the data unit created from each layer. In the top three layers, the data unit (packets) should not be changed by any router or link-layer switch. In the bottom two layers, the packet created by the host is changed only by the routers, not by the link-layer switches.

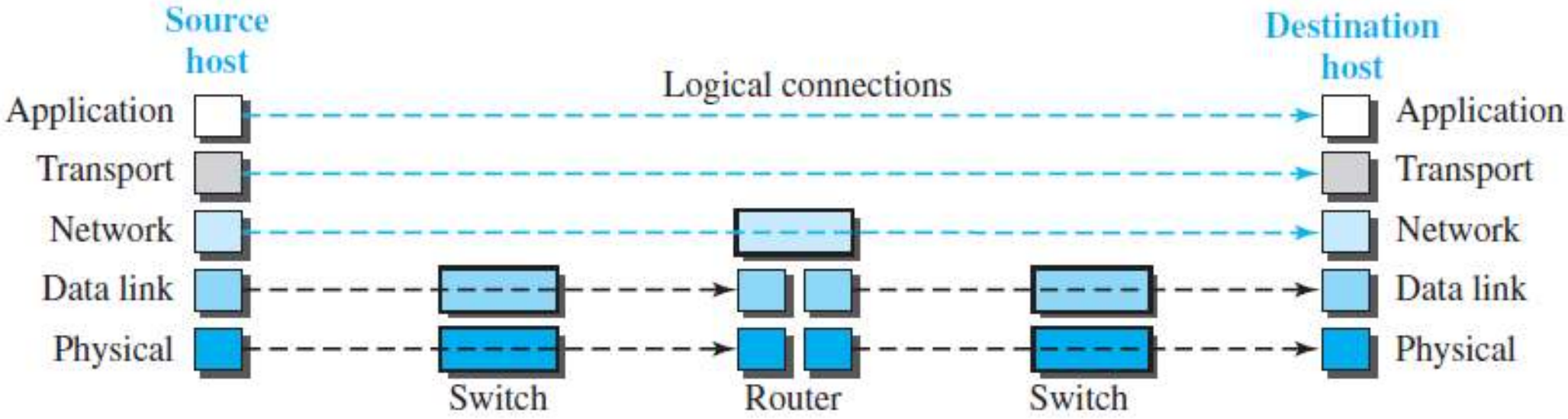
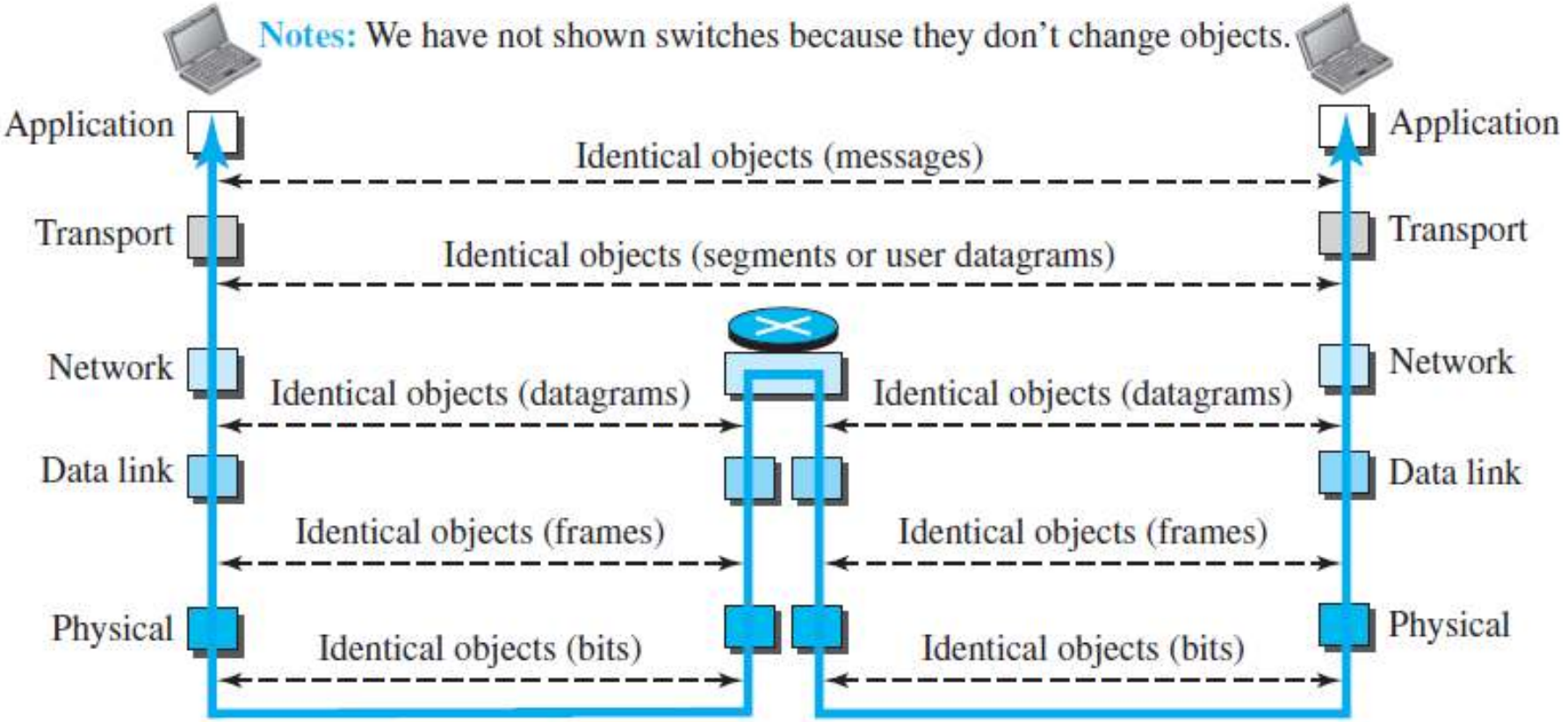


Figure 2.7 shows the second principle discussed previously for protocol layering. We show the identical objects below each layer related to each device.

Figure 2.7 *Identical objects in the TCP/IP protocol suite*



Note that, although the logical connection at the network layer is between the two hosts, we can only say that identical objects exist between two hops in this case because a router may fragment the packet at the network layer and send more packets than received. Note that the link between two hops does not change the object.

Layers in the TCP/IP Protocol Suite

Figure 2.6 *Logical connections between layers of the TCP/IP protocol suite*

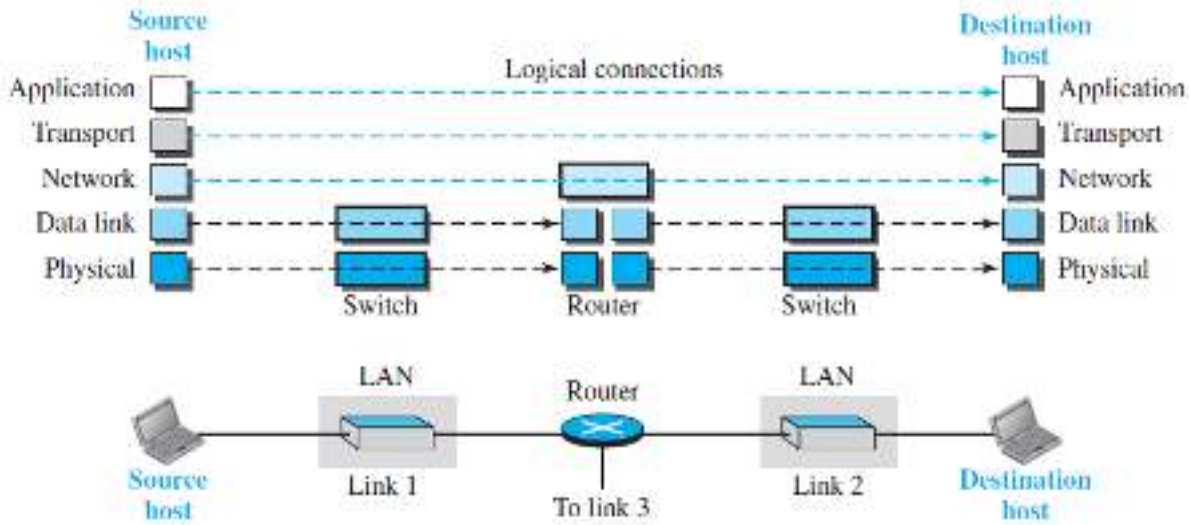
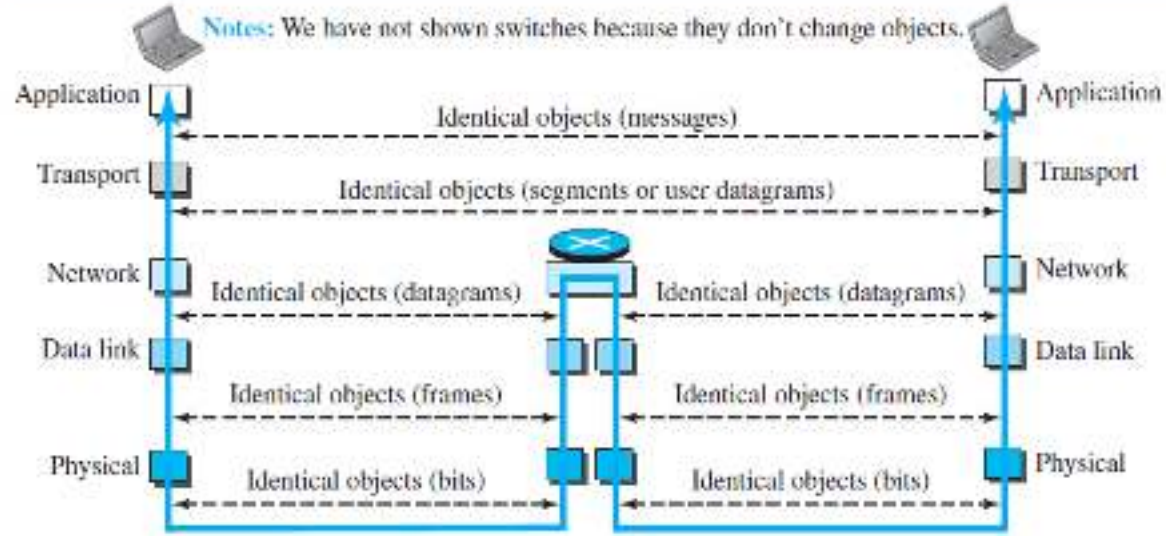


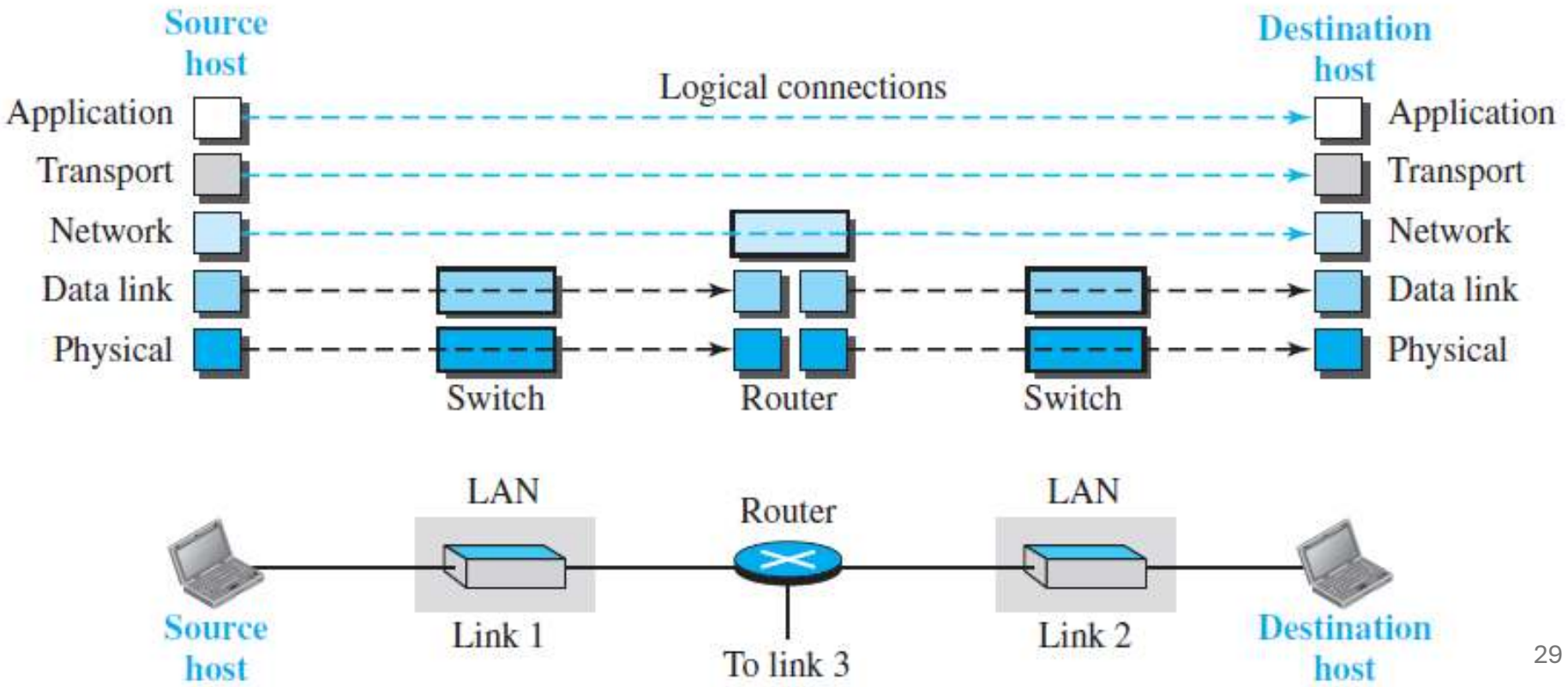
Figure 2.7 *Identical objects in the TCP/IP protocol suite*



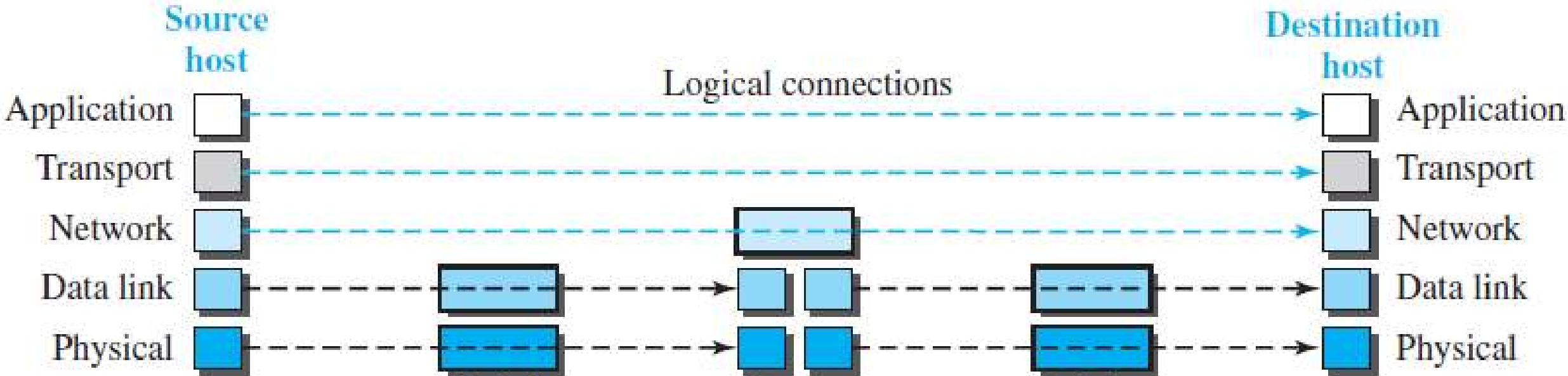
2.2.2 Layers in the TCP/IP Protocol Suite (Back to logical connection)

After the above introduction, we briefly discuss the functions and duties of layers in the TCP/IP protocol suite. Each layer is discussed in detail in the next five parts of the book. To better understand the duties of each layer, we need to think about the logical connections between layers. Figure 2.6 shows logical connections in our simple internet.

Figure 2.6 *Logical connections between layers of the TCP/IP protocol suite*



Logical connections help us better understand each layer's responsibilities. As shown in the figure, the application, transport, and network layers operate end-to-end, while the data-link and physical layers function hop-to-hop, where a hop is a host or router. In other words, the top three layers serve the internet, whereas the lower two layers operate within the link domain.



Another way to understand logical connections is by considering the data unit created at each layer. In the top three layers, packets remain unchanged by routers or link-layer switches. In the bottom two layers, packets generated by the host are modified only by routers, not by link-layer switches.

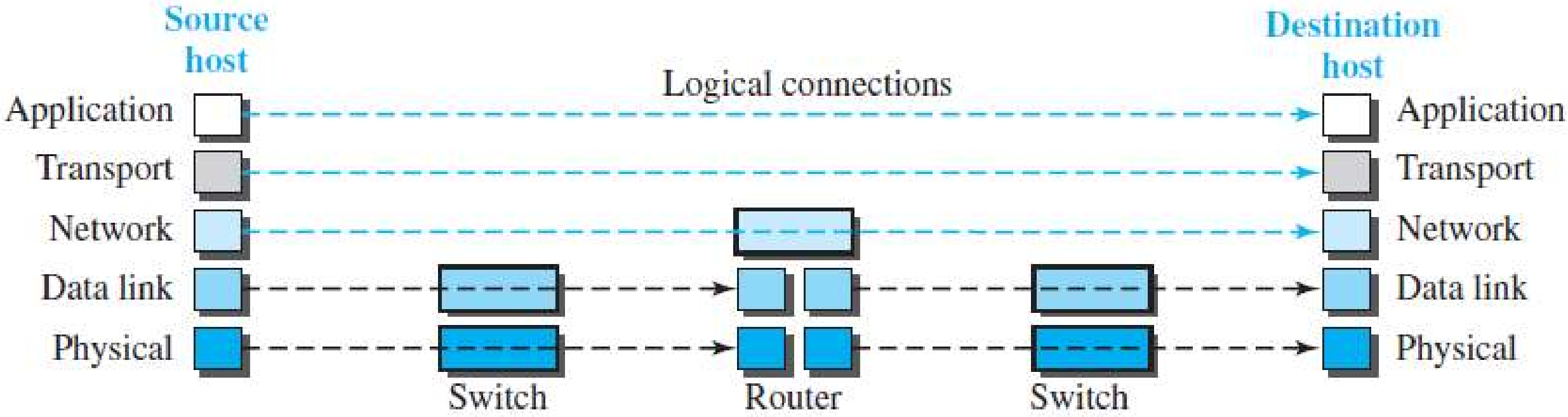
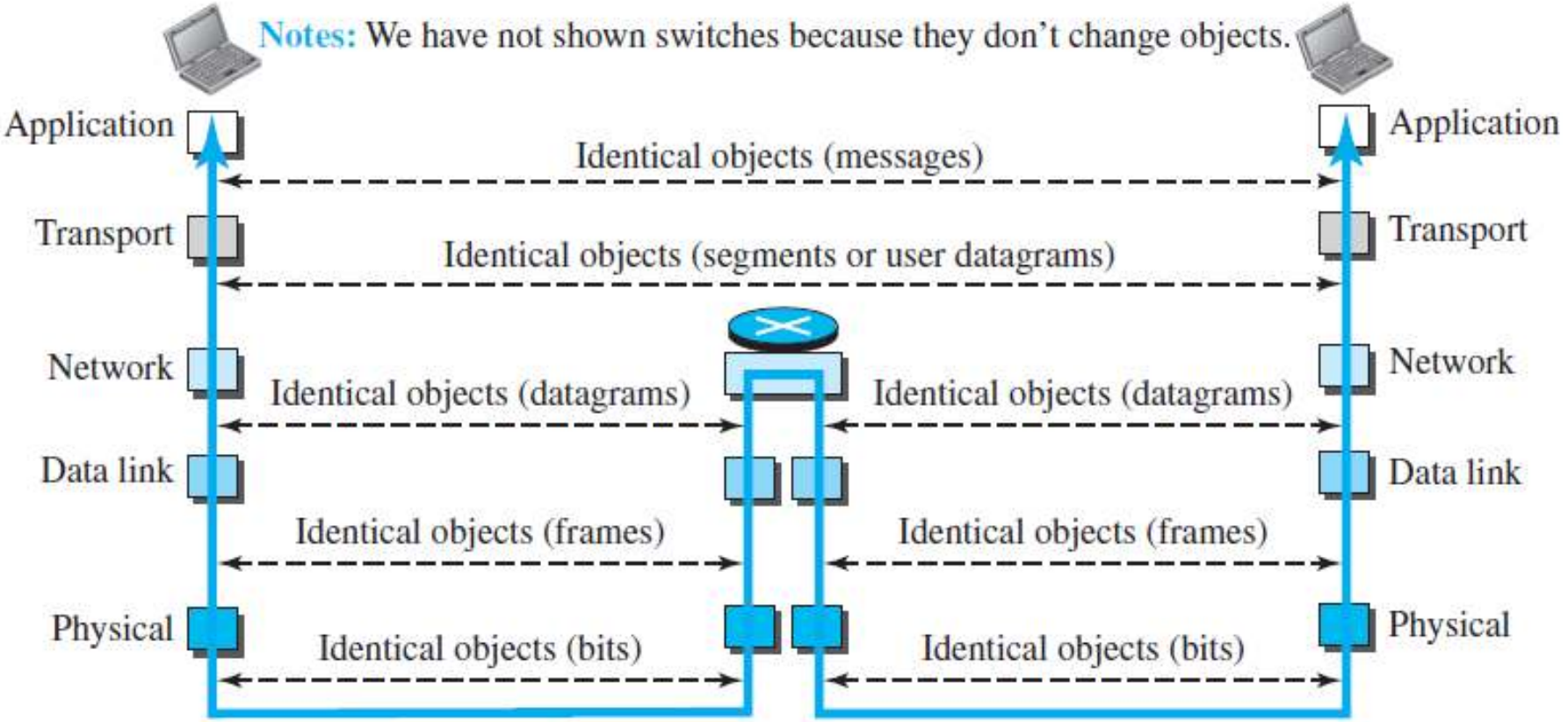


Figure 2.7 shows the second principle discussed previously for protocol layering. We show the identical objects below each layer related to each device.

Figure 2.7 *Identical objects in the TCP/IP protocol suite*



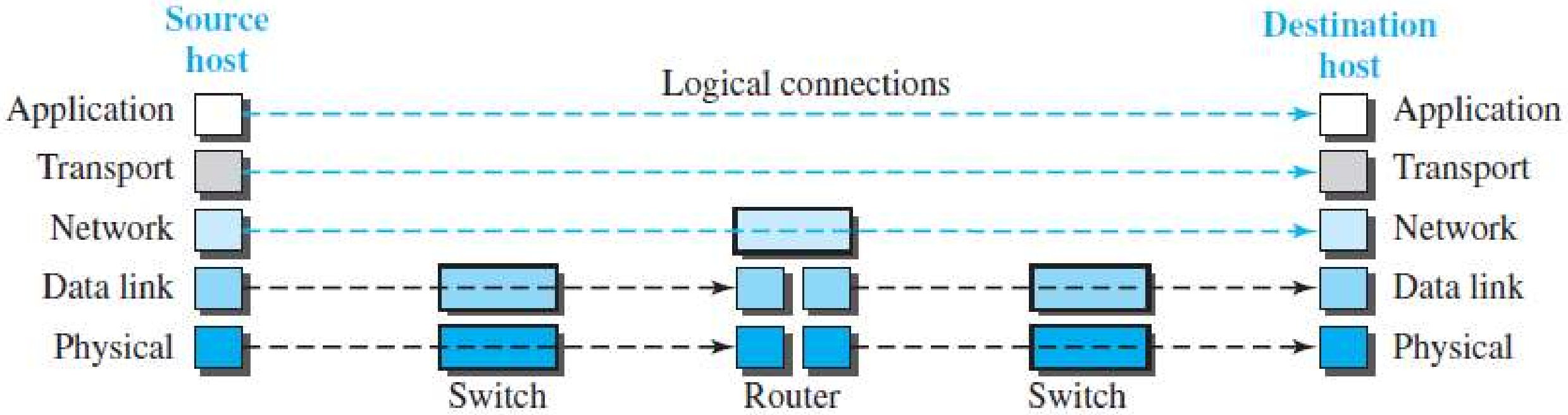
Note that, although the logical connection at the network layer is between the two hosts, we can only say that identical objects exist between two hops in this case because a router may fragment the packet at the network layer and send more packets than received. Note that the link between two hops does not change the object.

2.2.3 Description of Each Layer

After understanding the concept of logical communication, we are ready to briefly discuss the duty of each layer. Our discussion in this chapter will be very brief, but we come back to the duty of each layer in next five parts of the book.

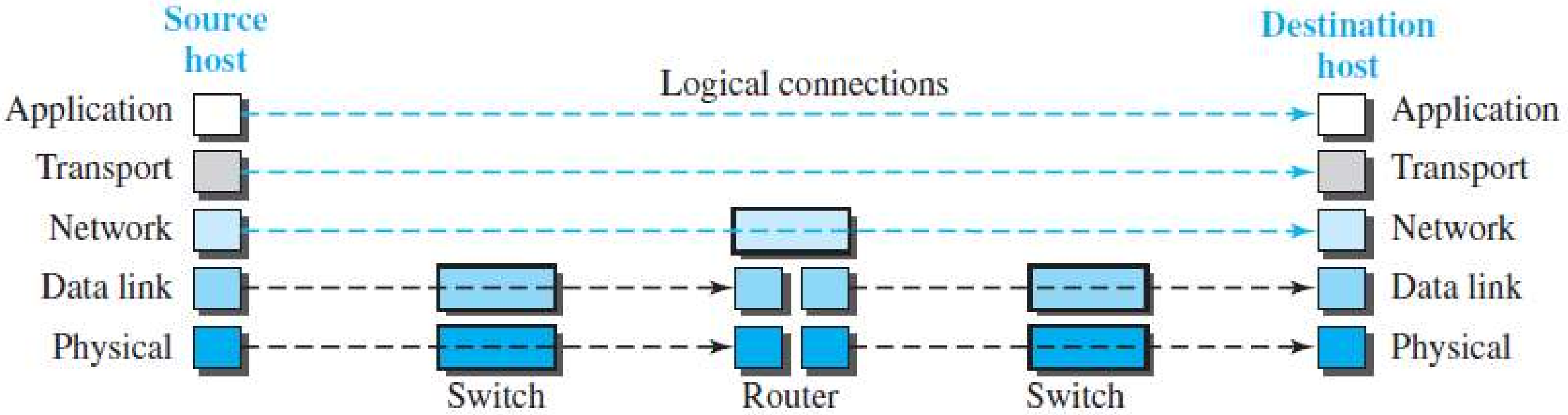
Physical Layer

The physical layer transmits individual bits in a frame across the link. Though it's the lowest layer, communication at this level is still logical, as the transmission medium (cable or air) carries electrical or optical signals, not bits. Several protocols convert bits to signals, which are discussed in Part II.



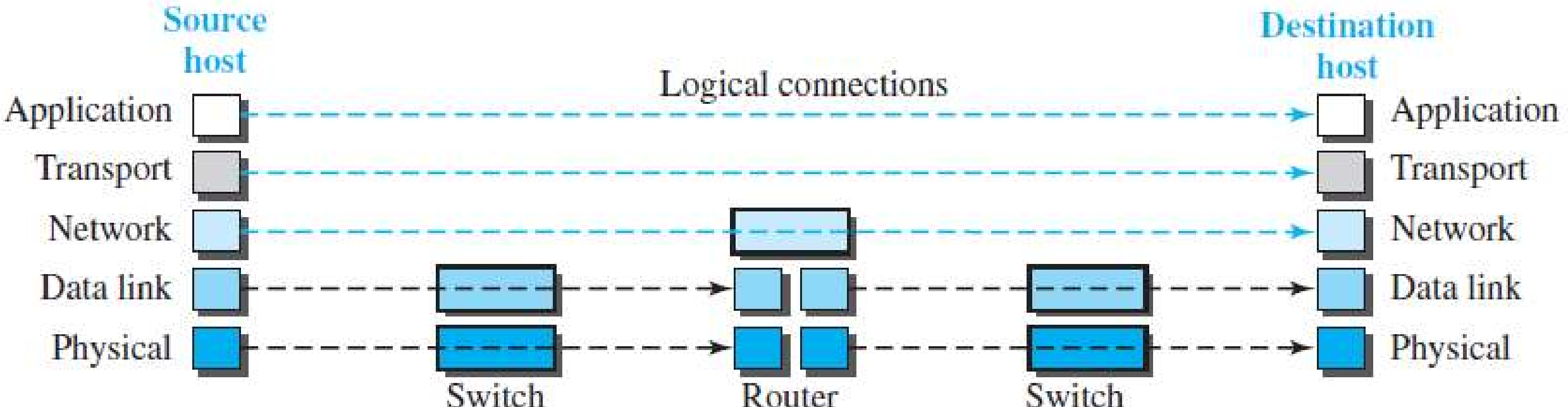
Data-link Layer

An internet consists of links (LANs and WANs) connected by routers, which determine the best path for a datagram. The data-link layer is responsible for moving the datagram across the link, which can be wired or wireless, using various protocols. TCP/IP supports all standard and proprietary protocols. The data-link layer encapsulates the datagram into a frame, with some protocols offering error detection and correction.



Network Layer

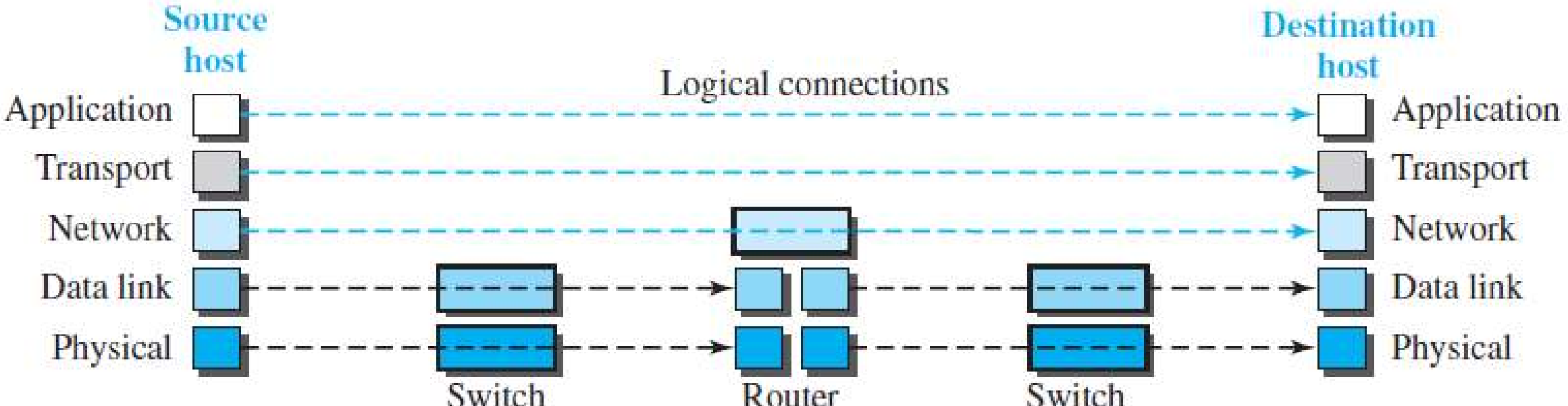
The network layer ensures host-to-host communication and handles routing packets through various routers. This separation of duties allows routers to operate without needing the application or transport layers. The main protocol at this layer, IP, defines the packet format (datagram) and address structure, while also managing packet routing from source to destination.



Network Layer

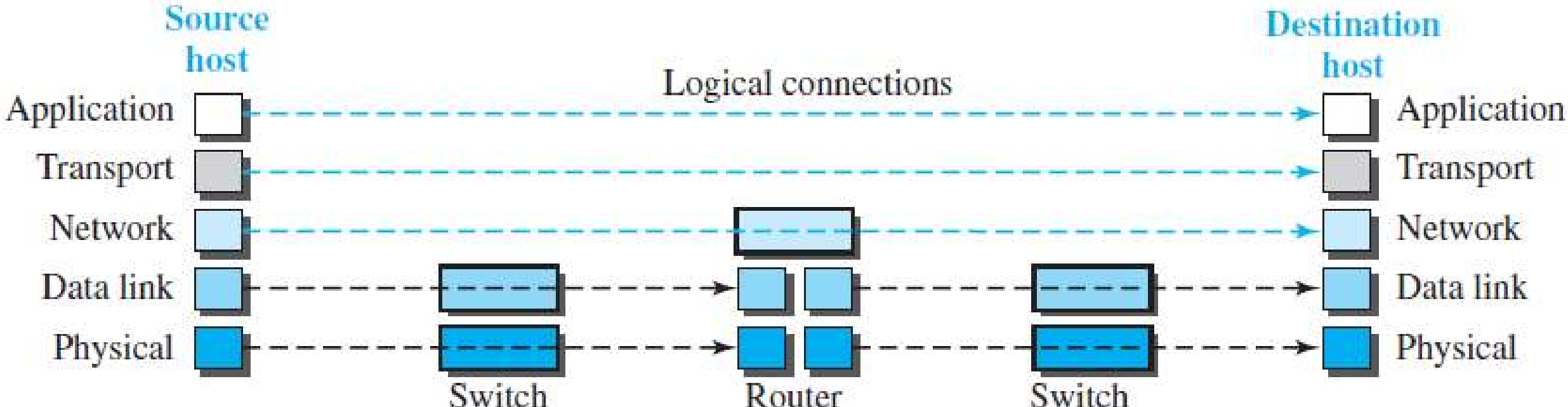
IP is a connectionless protocol that lacks flow, error, and congestion control, relying on the transport layer for these services. The network layer includes unicast and multicast routing protocols, which help create forwarding tables for routers. Auxiliary protocols like ICMP assist with error reporting, IGMP aids in multicast tasks, DHCP helps assign network addresses, and ARP finds link-layer addresses using network-layer addresses.

The Internet Control Message Protocol (ICMP)
The Internet Group Management Protocol (IGMP) is
The Dynamic Host Configuration Protocol (DHCP)
The Address Resolution Protocol (ARP)



Transport Layer

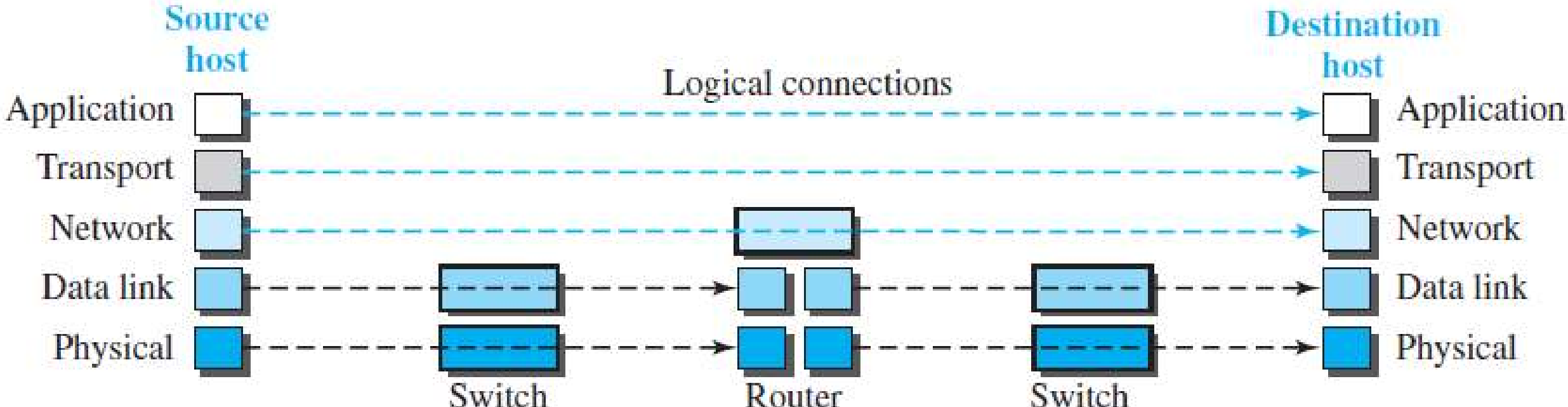
The transport layer provides end-to-end communication between source and destination hosts, encapsulating messages from the application layer into transport-layer packets (segments or user datagrams). It delivers messages to the corresponding application program at the destination. The need for a separate transport layer is due to task separation, ensuring it remains independent of the application layer. Multiple transport-layer protocols allow applications to choose the best one for their needs.



Transport Layer

The main transport-layer protocol, TCP, is connection-oriented and establishes a logical connection between two hosts before transferring data. It creates a logical pipe for byte streams and provides flow control, error control, and congestion control to ensure reliable data transfer without overwhelming the destination or losing segments.

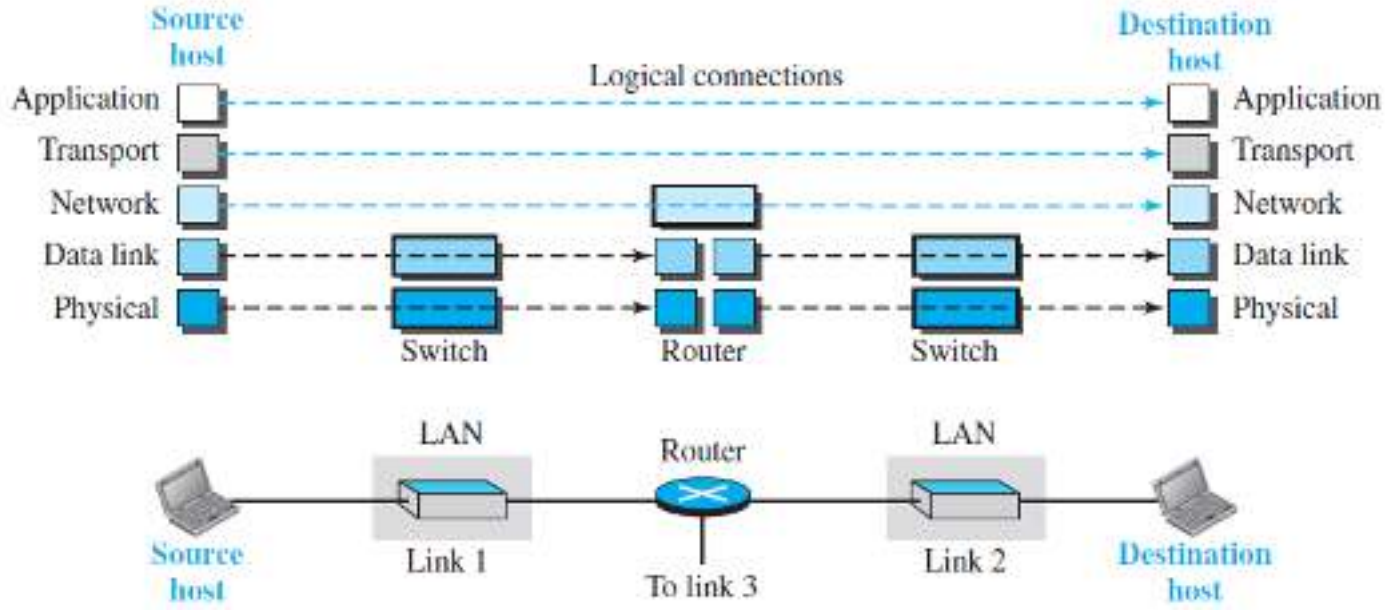
User Datagram Protocol(UDP) is a connectionless protocol that sends independent user datagrams without establishing a logical connection. It lacks flow, error, and congestion control, making it lightweight and ideal for applications needing fast transmission of short messages without the overhead of retransmissions. Stream Control Transmission Protocol SCTP, a newer protocol, is designed for emerging multimedia applications.



Application Layer

As shown in Figure 2.6, the logical connection between the two application layers is end-to-end, allowing them to exchange messages as if directly connected. However, communication passes through all layers. At the application layer, communication occurs between two processes (programs running at this layer). One process sends a request, while the other responds, making process-to-process communication the core function of this layer. The Internet's application layer includes many predefined protocols, but users can also create custom processes to run on both hosts.

Figure 2.6 *Logical connections between layers of the TCP/IP protocol suite*



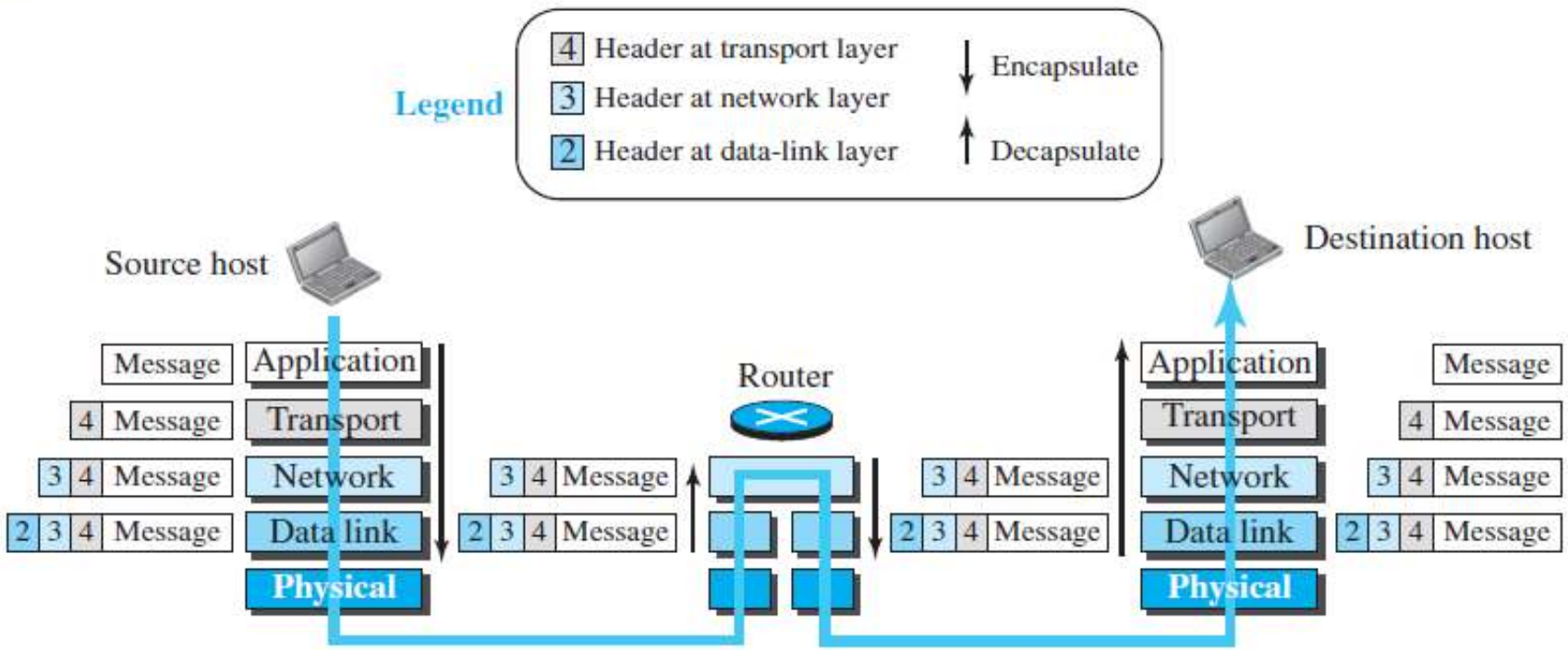
The Hypertext Transfer Protocol (HTTP) is a vehicle for accessing the World Wide Web (WWW). The Simple Mail Transfer Protocol (SMTP) is the main protocol used in electronic mail (e-mail) service. The File Transfer Protocol (FTP) is used for transferring files from one host to another. The Terminal Network (TELNET) and Secure Shell (SSH) are used for accessing a site remotely. The Simple Network Management Protocol (SNMP) is used by an administrator to manage the Internet at global and local levels. The Domain Name System (DNS) is used by other protocols to find the network-layer address of a computer. The Internet Group Management Protocol (IGMP) is used to collect membership in a group.

HTTP enables web access, SMTP handles email, and FTP transfers files. TELNET and SSH allow remote access, while SNMP aids network management. DNS resolves network addresses, and IGMP manages group memberships.

2.2.4 Encapsulation and Decapsulation

One of the important concepts in protocol layering in the Internet is encapsulation/decapsulation. Figure 2.8 shows this concept for the small internet in Figure 2.5.

Figure 2.8 *Encapsulation/Decapsulation*

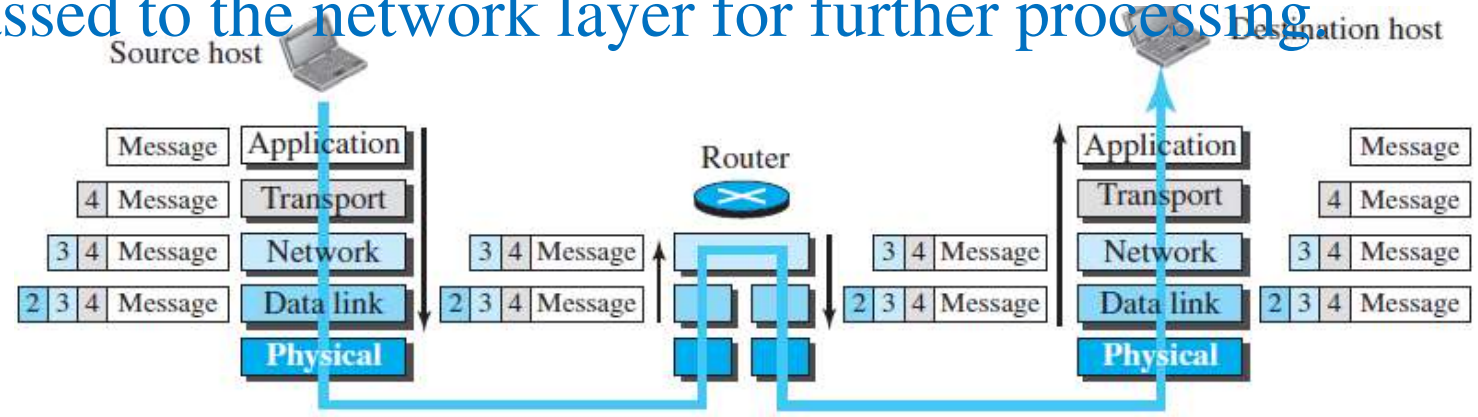


We have not shown the layers for the link-layer switches because no encapsulation/decapsulation occurs in this device. In Figure 2.8, we show the encapsulation in the source host, decapsulation in the destination host, and encapsulation and decapsulation in the router.

Encapsulation at the Source Host

At the source, we have only encapsulation

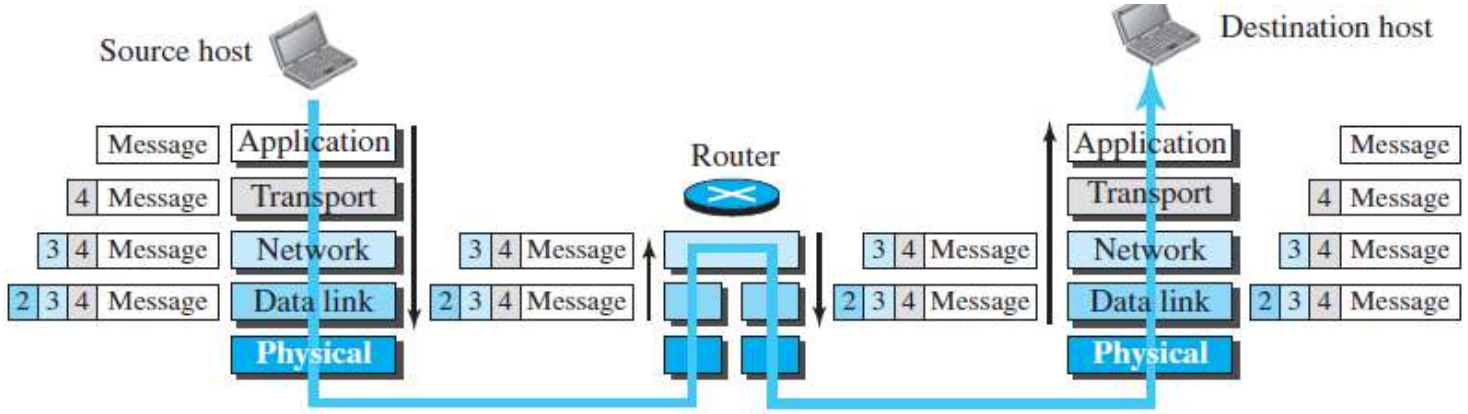
1. At the application layer, the exchanged data is called a message. Typically, a message does not include a header or trailer, but if present, the entire unit is still referred to as a message. The message is then passed down to the transport layer for further processing.
2. The transport layer treats the message as its payload, handling its reliable delivery. It adds a transport layer header containing source and destination application identifiers, along with essential information for end-to-end communication, such as flow control, error control, and congestion control. The resulting transport-layer packet is called a **segment** in TCP and a **user datagram** in UDP. This packet is then passed to the network layer for further processing.



Encapsulation at the Source Host

3-The **network layer** treats the transport-layer packet as its payload and adds its own header. This header includes source and destination host addresses, along with additional information for error checking, fragmentation, and other network functions. The resulting network-layer packet is called a **datagram**, which is then passed to the data-link layer.

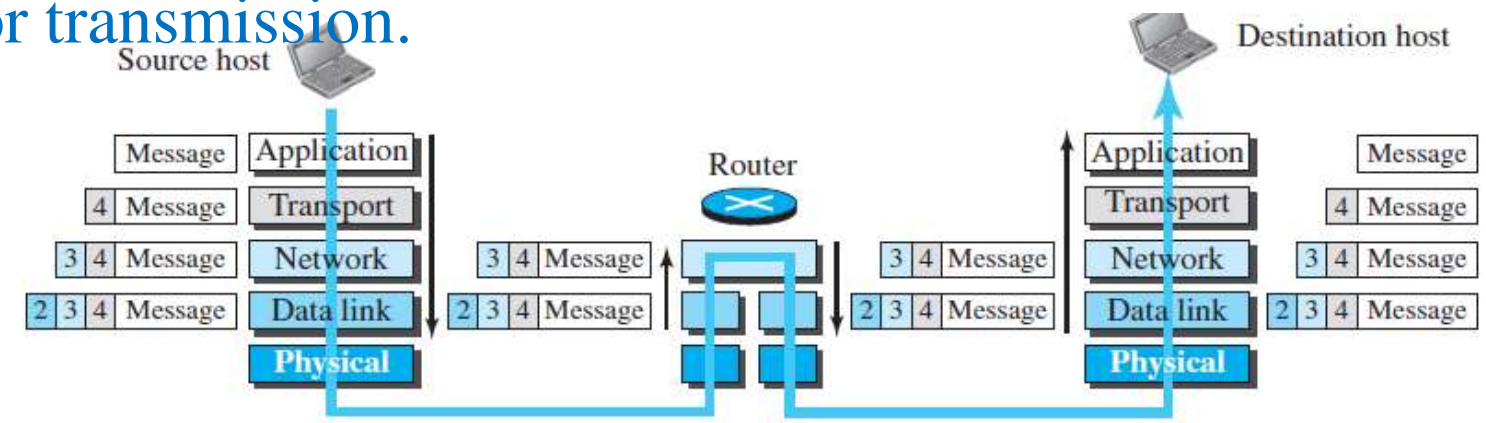
4-The **data-link layer** processes the datagram as its payload and appends its own header. This header contains link-layer addresses, identifying the host or the next hop (such as a router). The final link-layer packet, called a **frame**, is then transmitted to the physical layer for delivery.



Decapsulation and Encapsulation at the Router

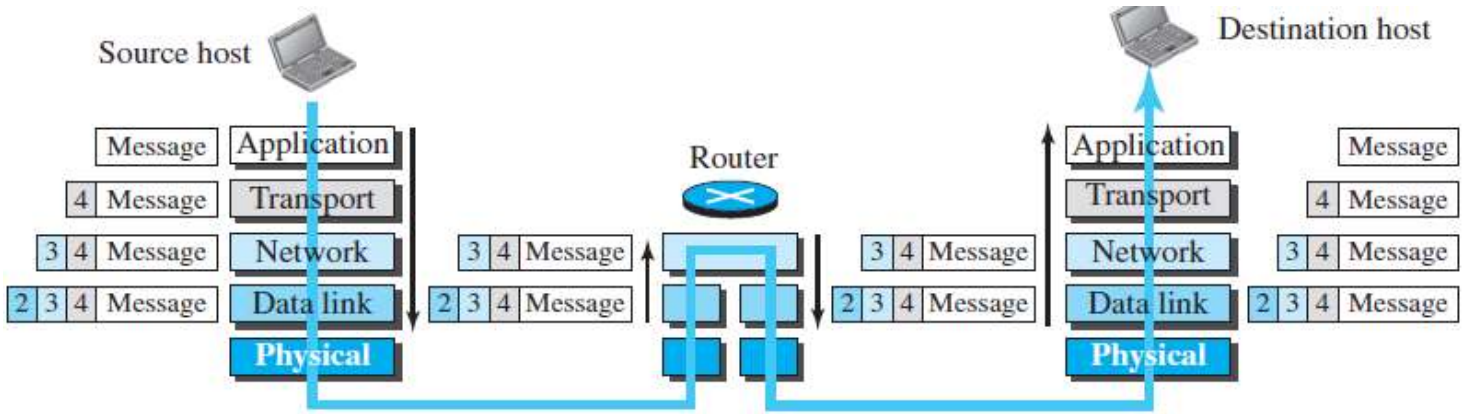
At the router, we have both decapsulation and encapsulation because the router is connected to two or more links.

1. Upon receiving the set of bits, the **data-link layer** decapsulates the **datagram** from the **frame** and forwards it to the **network layer**.
2. The **network layer** examines only the source, and destination addresses *in the datagram header* and refers to its forwarding table to determine the next hop. It does not modify the datagram contents unless fragmentation is required for transmission through the next link. The datagram is then passed to the **data-link layer** of the next link.
3. The **data-link layer** of the next link encapsulates the **datagram** into a **frame** and sends it to the **physical layer** for transmission.



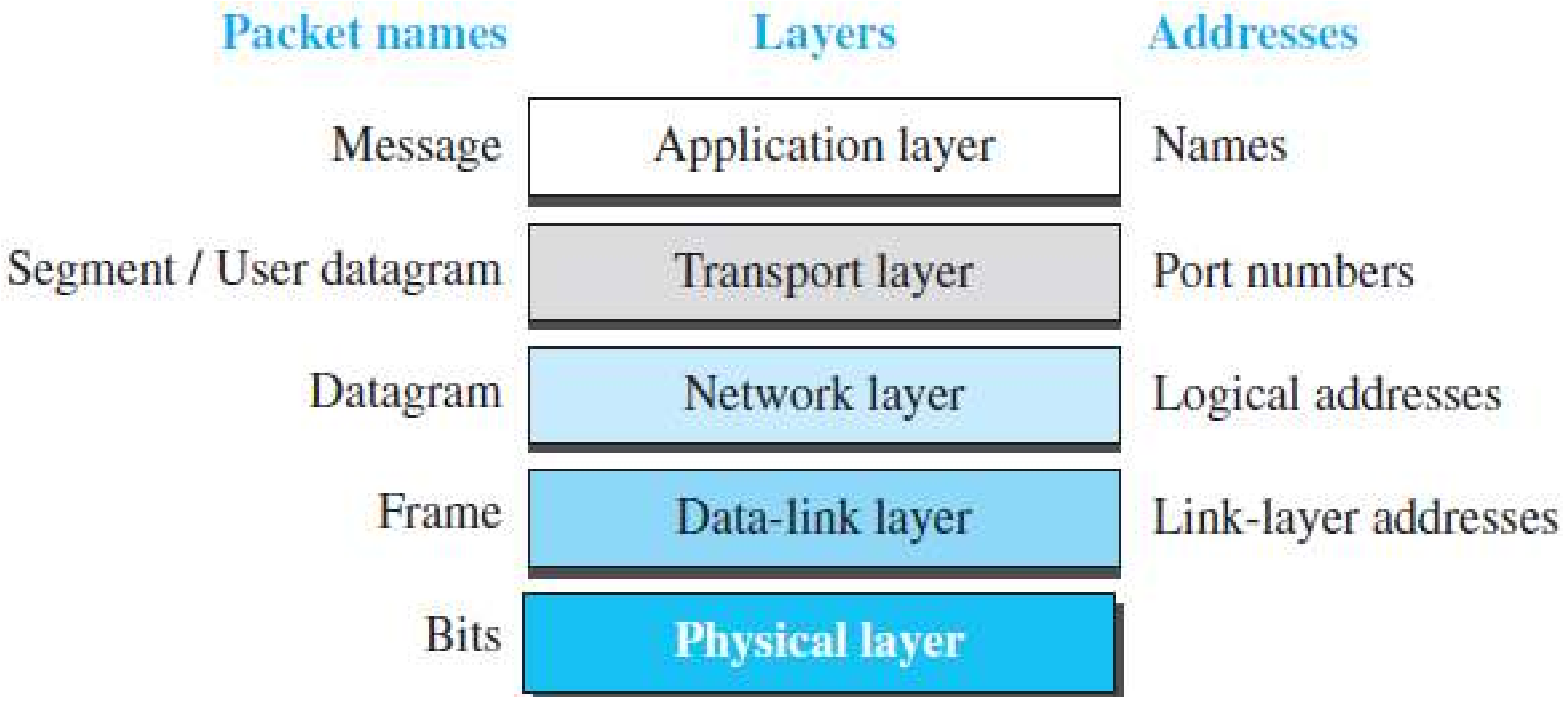
Decapsulation at the Destination Host

At the destination host, each layer only decapsulates the packet received, removes the payload, and delivers the payload to the next-higher layer protocol until the message reaches the application layer. It is necessary to say that decapsulation in the host involves error checking.



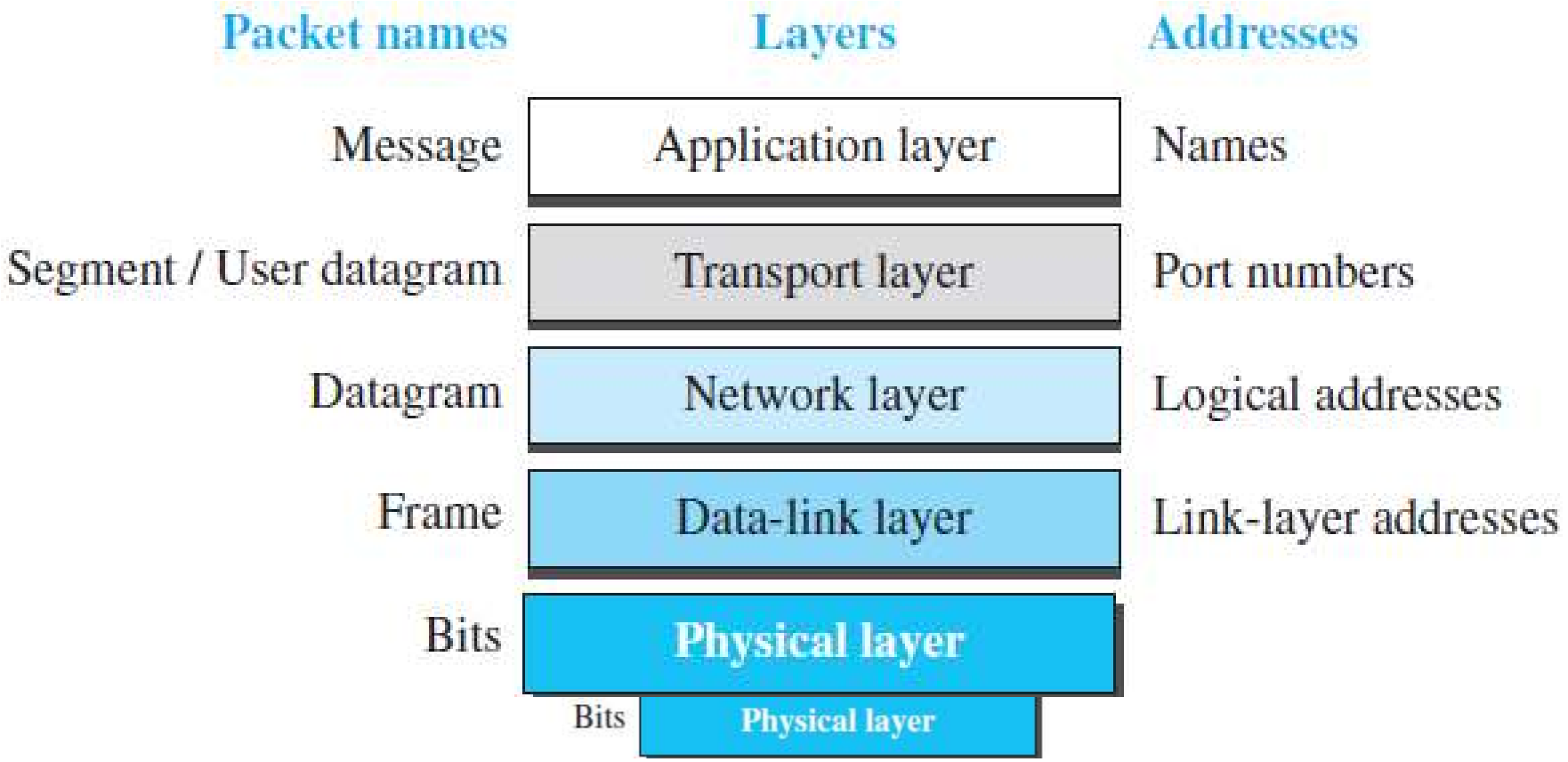
2.2.5 Addressing

In protocol layering, addressing is crucial for communication between layers. Each communication requires a source and destination address. While it may seem like five address pairs are needed, only four are typically used because the physical layer doesn't require addresses (as it deals with bits, not addressable units). Figure 2.9 shows the addressing at each layer.



Each layer in the protocol stack uses different types of addresses. At the application layer, addresses are typically names like website domains or email addresses. At the transport layer, port numbers identify specific application programs. At the network layer, global addresses uniquely define a device's connection to the Internet. The link-layer uses local MAC addresses to identify specific hosts or routers within a network.

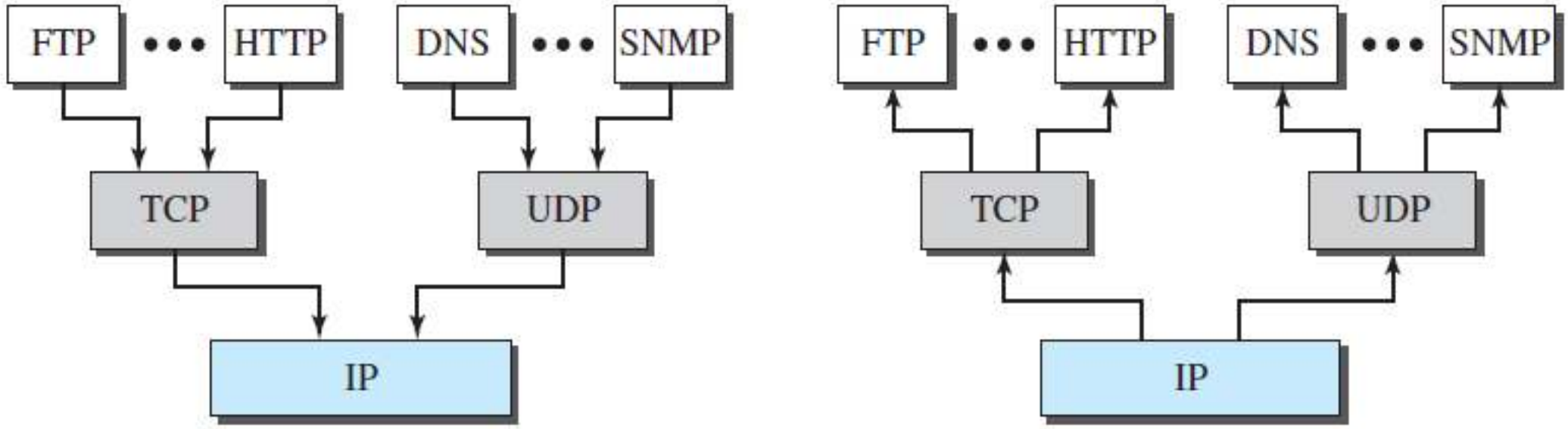
A MAC (Media Access Control)



2.2.6 Multiplexing and Demultiplexing

The TCP/IP protocol suite uses multiplexing and demultiplexing. Multiplexing allows a protocol to encapsulate packets from multiple higher-layer protocols (one at a time), while demultiplexing enables a protocol to decapsulate and deliver packets to the appropriate higher-layer protocol. This occurs at the transport, network, and application layers.

Figure 2.10 *Multiplexing and demultiplexing*

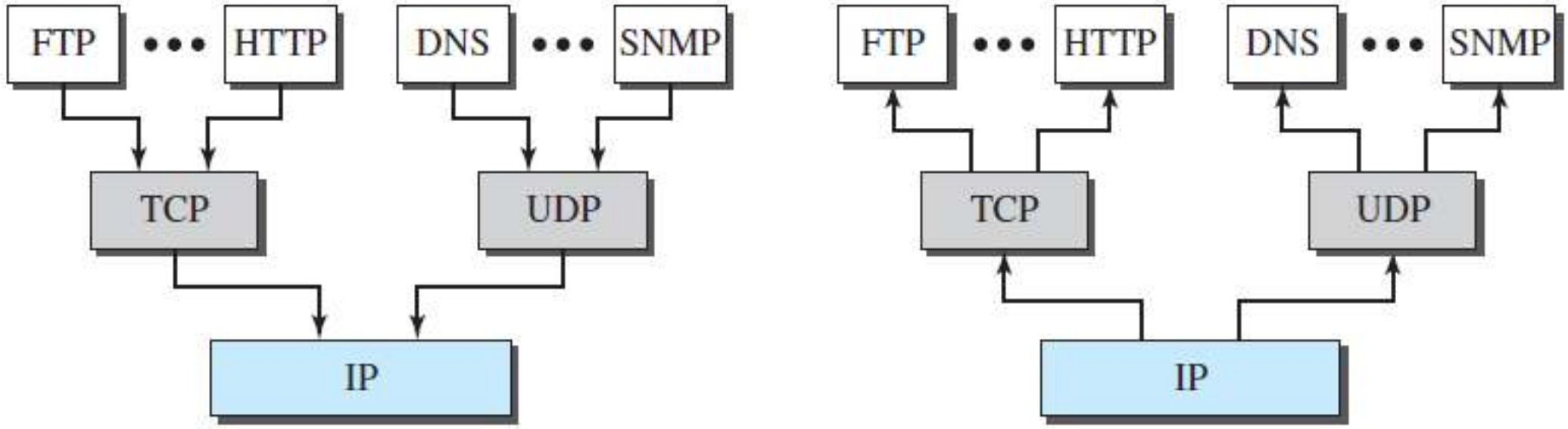


a. Multiplexing at source

b. Demultiplexing at destination

To be able to multiplex and demultiplex, a protocol needs to have a field in its header to identify to which protocol the encapsulated packets belong. At the transport layer, either UDP or TCP can accept a message from several application-layer protocols. At the network layer, IP can accept a segment from TCP or a user datagram from UDP. IP can also accept a packet from other protocols such as ICMP, IGMP, and so on. At the data-link layer, a frame may carry the payload coming from IP or other protocols such as ARP (see Chapter 9).

Figure 2.10 *Multiplexing and demultiplexing*



a. Multiplexing at source

b. Demultiplexing at destination

2.3 THE OSI MODEL

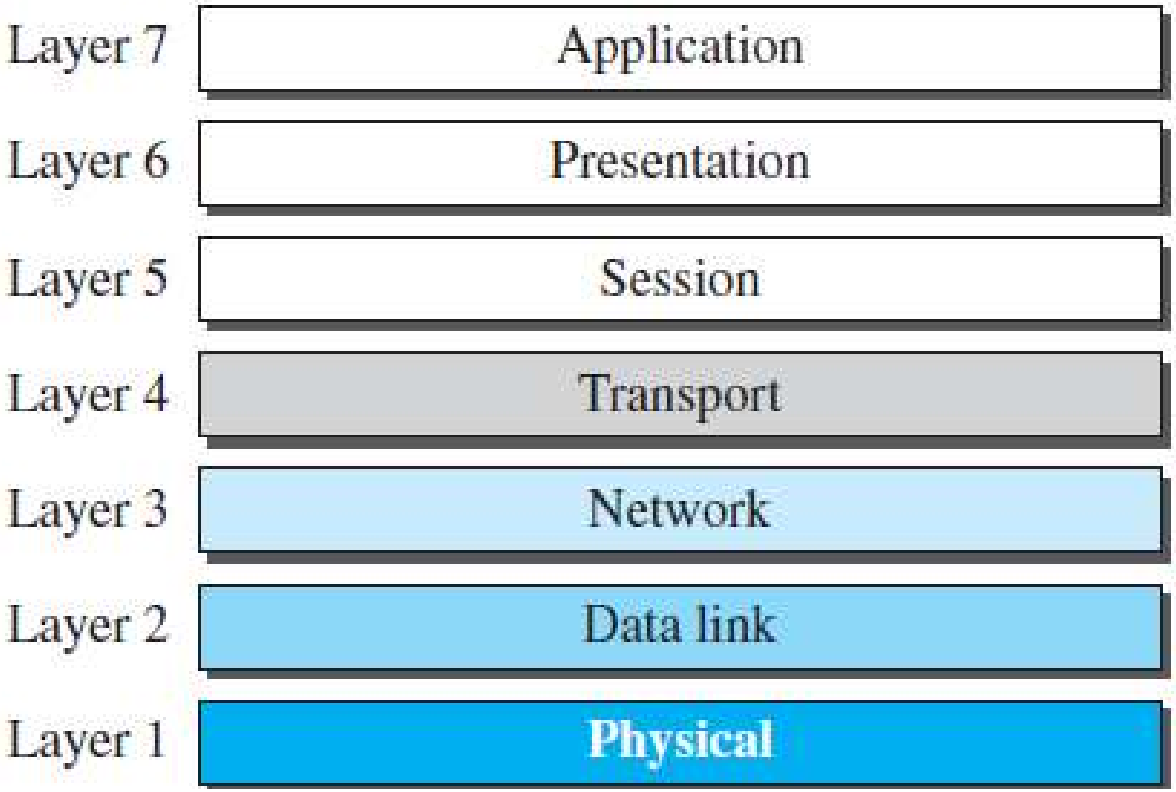
Although, when speaking of the Internet, everyone talks about the TCP/IP protocol suite, this suite is not the only suite of protocols defined. Established in 1947, the **International Organization for Standardization (ISO)** is a multinational body dedicated to worldwide agreement on international standards. Almost three-fourths of the countries in the world are represented in the ISO. An ISO standard that covers all aspects of network communications is the **Open Systems Interconnection (OSI) model.** It was first introduced in the late 1970s.

ISO is the organization; OSI is the model.

An *open system* is a set of protocols that allows any two different systems to communicate regardless of their underlying architecture. The purpose of the OSI model is to show how to facilitate communication between different systems without requiring changes to the logic of the underlying hardware and software. The OSI model is not a protocol; it is a model for understanding and designing a network architecture that is flexible, robust, and interoperable. The OSI model was intended to be the basis for the creation of the protocols in the OSI stack.

The OSI model is a layered framework for the design of network systems that allows communication between all types of computer systems. It consists of seven separate but related layers, each of which defines a part of the process of moving information across a network (see Figure 2.11).

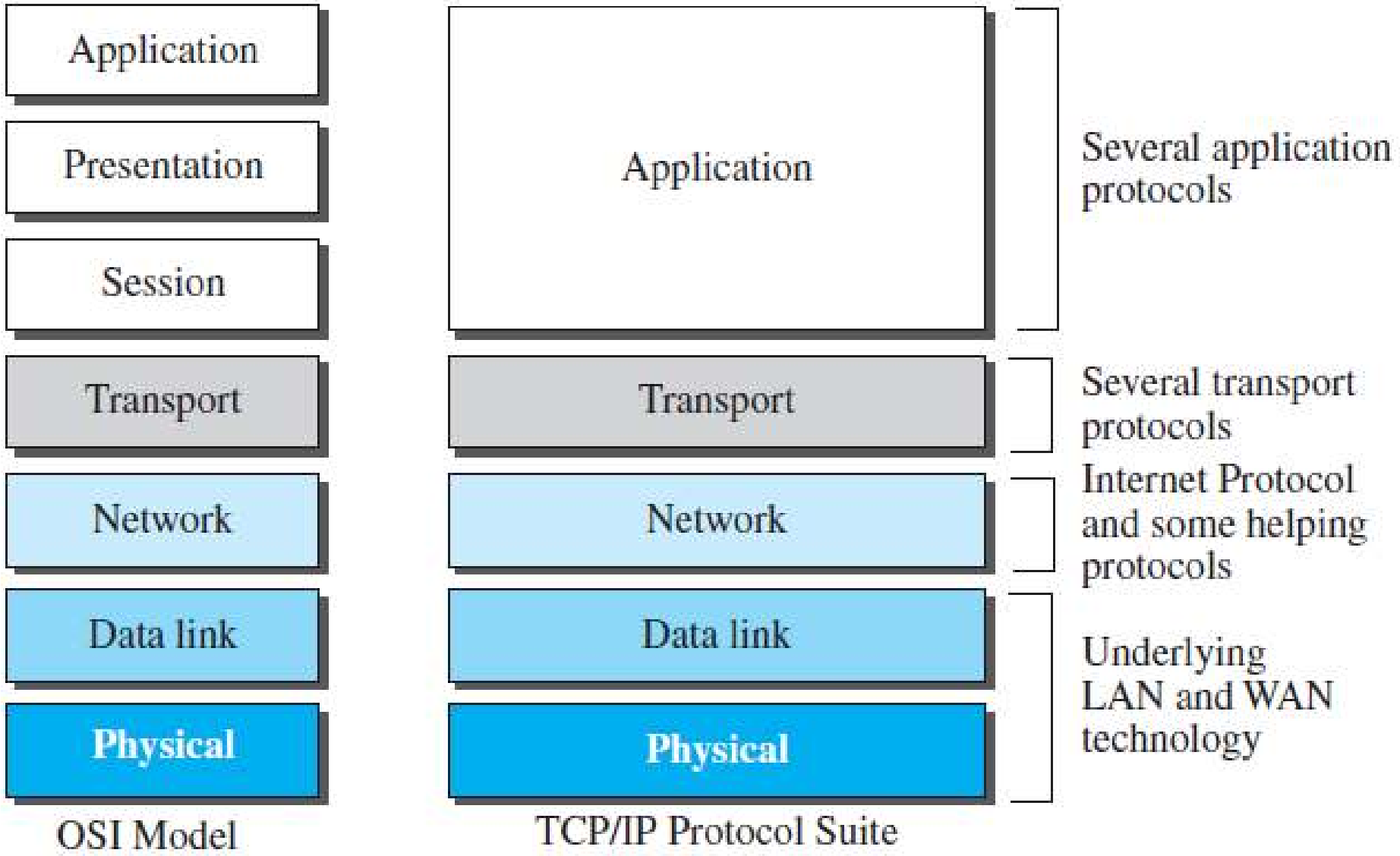
Figure 2.11 *The OSI model*



2.3.1 OSI versus TCP/IP

When we compare the two models, we find that two layers, session and presentation, are missing from the TCP/IP protocol suite. These two layers were not added to the TCP/IP protocol suite after the publication of the OSI model. The application layer in the suite is usually considered to be the combination of three layers in the OSI model, as shown in Figure 2.12.

Figure 2.12 *TCP/IP and OSI model*



Two reasons were mentioned for this decision. First, TCP/IP has more than one transport-layer protocol. Some of the functionalities of the session layer are available in some of the transport-layer protocols. Second, the application layer is not only one piece of software. Many applications can be developed at this layer. If some of the functionalities mentioned in the session and presentation layers are needed for a particular application, they can be included in the development of that piece of software.

2.3.2 Lack of OSI Model's Success

The OSI model appeared after the TCP/IP protocol suite. Most experts were at first excited and thought that the TCP/IP protocol would be fully replaced by the OSI model. This did not happen for several reasons, but we describe only three, which are agreed upon by all experts in the field. First, OSI was completed when TCP/IP was fully in place and a lot of time and money had been spent on the suite; changing it would cost a lot. Second, some layers in the OSI model were never fully defined. For example, although the services provided by the presentation and the session layers were listed in the document, actual protocols for these two layers were not fully defined, nor were they fully described, and the corresponding software was not fully developed. Third, when OSI was implemented by an organization in a different application, it did not show a high enough level of performance to entice the Internet authority to switch from the TCP/IP protocol suite to the OSI model.